GTCFN: A Graph-Based Transformer and Convolution Fusion Network for Hyperspectral Image Classification

Xiaofeng Zhao, Junyi Ma, Lei Wang, Jiayi Shi, Yao Ding, Zhili Zhang, and Jie Feng, *Senior Member, IEEE*

***Abstract*—Graph neural networks (GNNs) are capable of modeling complex non-Euclidean structures through information transfer, and, thus, have been widely used in the field of hyperspectral image (HSI) classification. However, conventional GNNs often have difficulty in handling regular grid data, which in turn loses positional information or spatial coherence, as well as in capturing long-range dependencies, which affects their performance in heterogeneous and limited-sample conditions. To address these limitations, this article proposes a novel graph-based Transformer and convolution fusion network (GTCFN) that integrates the local representation power of convolutional neural networks (CNNs) with the global reasoning capability of graph-based Transformers. GTCFN consists of two synergistic branches: a graph transformer subnetwork (GTsN) that models high-level semantic structures among superpixels via attention-based topology learning, and a spectral–spatial convolutional subnetwork (S^2CsN) that extracts multiscale fine-grained features using 5×5, 7×7, and 9×9 convolutional kernels. To enhance efficiency and generalization, GTCFN incorporates kernelized attention with random feature mapping, reducing the complexity from $\mathcal{O}(M^2)$ to $\mathcal{O}(M)$. At the same time, attention over-smoothing is avoided by introducing a Gumbel-based multihead random aggregation mechanism. Experiments conducted on four benchmark datasets, namely, Indian Pines (IP), Pavia University (PU), Salinas (SA), and WHU-Hi-HongHu (HongHu), show that GTCFN achieves state-of-the-art performance with OA of 95.62%, 98.34%, 97.88%, and 96.69%, which is significantly better than 12 other algorithms, such as CNNs, graph-based models, and hybrid network models. The core code for GTCFN is posted on https://github.com/Majunyi310321/GTCFN**

***Index Terms*—Convolutional network, graph neural network (GNN), hyperspectral image (HSI) classification, image classification, Transformer.**

NOMENCLATURE

$\mathcal{G} = \langle \mathcal{V}, \varepsilon \rangle$	Graph.
$\mathcal{V}$	Set of nodes.
$\mathcal{E}$	Set of edges.
$\mathbf{x}_i$	Feature vector associated with node i.
$\mathbf{z}^l$	Node representations at layer l.
a_{ij}	Adjacency matrix.
W	Weight matrix.
Q	Pixel-to-superpixel association matrix.
$\phi(\cdot)$	Kernel mapping function (e.g., exponential).
τ	Temperature scaling factor.
$\kappa(\cdot, \cdot)$	Kernel function for similarity [e.g., radial basis function (RBF)].
$\odot$	Hadamard (elementwise) product.
$\tilde{G}_k$	Expanded weight matrix for Gumbel multihead random aggregation (GMRA).
H	Number of attention heads.
K	Number of sampled aggregators.

Received 22 June 2025; revised 14 September 2025 and 28 September 2025; accepted 5 October 2025. Date of publication 7 October 2025; date of current version 3 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 41404022 and in part by the National Foundation for Enhancing Fundamental Science in China under Grant 2021-JCJQ-JJ-0871. *(Corresponding authors: Junyi Ma; Yao Ding.)*

Xiaofeng Zhao, Junyi Ma, Lei Wang, Jiayi Shi, Yao Ding, and Zhili Zhang are with Xi'an Research Institute of High Technology, Xi'an 710025, China (e-mail: 15029904066@163.com; dingyao.88@outlook.com).

Jie Feng is with the Key Laboratory of Intelligent Perception and Image Understanding, Ministry of Education of China, Xidian University, Xi'an 710025, China.

Digital Object Identifier 10.1109/TGRS.2025.3618962

I. INTRODUCTION

HYPERSPECTRAL image (HSI) technology enables the simultaneous acquisition of images across dozens to hundreds of contiguous and finely resolved spectral bands, thereby integrating both spectral and spatial information [1]. This capability has provided a highly detailed foundation for classification tasks in various application domains, such as environmental monitoring [2], precision agriculture [3], biomedical analysis [4], geological exploration [5], and defense-related operations [6]. However, with the continuous improvement in spectral resolution, hyperspectral data have become increasingly information-rich, which introduces several challenges. These include the growing complexity of model architectures, escalating computational costs [7], limited ability to process unstructured data, and significant difficulties in accurately extracting joint spatial–spectral features [8]. To overcome these obstacles, the extensive research has been conducted across multiple technical layers and interdisciplinary fields.

Early studies on HSI classification (HSIC) primarily relied on simple spectral feature fusion techniques and traditional pattern recognition methods. A common approach was the use of a two-stage framework. First, the dimensionality reduction was applied to extract representative features using techniques, such as principal component analysis (PCA) [9], linear discriminant analysis [10], and kernel PCA (KPCA) [11]. These extracted features were subsequently fed into conventional

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

classifiers such as decision trees [12], random forests [13], k-nearest neighbors [14], support vector machines (SVMs) [15], and extreme learning machines [16] for final classification. Although these methods alleviated some of the challenges associated with high-dimensional data, they heavily depended on handcrafted feature engineering and demonstrated limited generalization capabilities.

The rapid development of deep learning has introduced new paradigms for HSIC. Representative models include deep belief networks [17], autoencoders [18], recurrent neural networks [19], and convolutional neural networks (CNNs) [20]. Compared with traditional HSIC techniques, deep learning-based methods can automatically learn robust and adaptive hierarchical features from labeled data, thereby significantly improving both classification accuracy and generalization performance. Among various deep learning architectures, CNNs have emerged as the most widely adopted in HSIC, due to their core advantages of local connectivity, weight sharing, and translation invariance. Hu et al. [21] were among the first to apply CNNs to HSIC, utilizing a 1D-CNN to extract spectral features. Subsequently, Hamida et al. [22] proposed a 3D-CNN to jointly capture the spectral and spatial information. To further enhance the depth and feature abstraction, Zhong et al. [23] introduced residual connections into 3D-CNN architectures, resulting in the spectral–spatial residual network (SSRN), which enables more effective deep feature learning. Building upon this, Roy et al. [24] developed a hybrid architecture [hybrid spectral–spatial network (HybridSN)] that integrates 3D-CNN modules for low-level spectral–spatial extraction with 2D-CNN layers for high-level feature refinement. Nevertheless, CNNs face inherent limitations in modeling long-range dependencies due to their localized receptive fields. Although deeper networks can expand the receptive field by stacking additional layers, this strategy often results in substantially increased network depth and computational overhead.

In contrast, Transformer-based approaches leverage self-attention mechanisms to effectively capture long-range dependencies and global contextual information in HSI data. To introduce the Transformer architecture into computer vision, Alexey [25] proposed the Vision Transformer (ViT). To improve the computational efficiency, Zhang et al. [26] introduced a lightweight Transformer (LiT), which reduces model parameters and mitigates overfitting through optimized architectural design. Zhang et al. [27] proposed a multiview Transformer to address spatial overfitting by leveraging complementary spatial perspectives. Thanks to the excellent scalability of Transformers, they can also be flexibly integrated with other neural architectures. For instance, Cheng et al. [28] designed a dual-branch feature fusion (DBFF) module that combines CNNs with Transformer blocks to jointly capture spatial–spectral information at multiple scales. Similarly, Yang et al. [29] proposed a hybrid model that combines the spatial representation capabilities of graph convolutional networks (GCNs) with the global contextual modeling strength of Transformers to address the limited integration of spatial and spectral information in existing models. Despite these advances, most current Transformer-based methods still primarily focus on Euclidean spatial relationships, making it challenging to capture non-Euclidean or topological structures inherent in hyperspectral data.

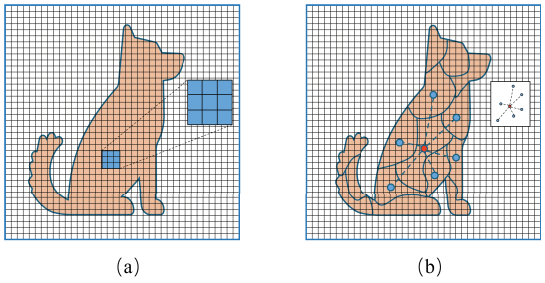


Fig. 1. Illustration of CNN and GCN architectures. The orange region denotes the target object to be classified. (a) Convolutional operation in CNNs: the blue rectangular area indicates the local receptive field, which extracts spatial features through a sliding window mechanism and generates pixel-level representations. (b) Graph convolutional operation in GCNs: based on the superpixel segmentation, each node corresponds to a superpixel centroid, while edges represent spatial adjacency. This graph-based structure allows GCNs to perform effective convolution over irregularly shaped regions.

In recent years, graph neural networks (GNNs) have gained increasing attention in HSIC due to their remarkable ability to model non-Euclidean data structures [30]. By representing pixels or superpixels as graph nodes and defining edges based on the spectral similarity and spatial proximity, GNNs can effectively capture complex, nonlinear relationships within hyperspectral data. Various GNN architectures have been proposed for HSIC, including GCNs, graph recurrent networks [31], graph autoencoders [32], and graph adversarial networks [33]. Among them, GCN is most widely used in the field of image classification. Early studies predominantly adopted pixelwise graph construction strategies. For instance, Shahraki and Prasad [34] were among the first to introduce GCNs into HSIC, facilitating low-dimensional feature representation and classification. Building on this, Qin et al. [35] enhanced the feature expressiveness by integrating both spectral and spatial cues during graph construction. Subsequent developments, such as nonlocal graph convolution [36], ARMA filtering [37], and multimodal feature fusion [38], have further strengthened the contextual sensitivity and discriminative capacity of GNN-based models. To reduce the computational burden, superpixel-based graph construction has emerged as a promising alternative. These approaches apply image oversegmentation to reduce the number of graph nodes while preserving essential feature information. For example, the context-aware dynamic GCN (CAD-GCN) proposed by Wan et al. [39] automates pixel-to-superpixel mapping and improves classification performance in spectrally heterogeneous regions. The inherent scalability of GNNs has also enabled their integration with other deep learning frameworks to enhance the feature complementarity and address the limitations of individual models. Liu et al. [40] developed a hybrid CNN–GCN architecture that fuses pixel-level and superpixel-level features, thereby preserving fine-grained detail while capturing broader spatial context. Likewise, the attention multihop graph and multiscale convolutional

fusion network (AMGCFN) proposed by Zhou et al. [41] employs parallel CNN and GCN branches for hierarchical feature extraction. Recent innovations include HGCN-MHF [42], which integrates CNNs with hypergraph convolution to exploit high-order topological information, and the prototype-enhanced GCN by Ding et al. [43], which stabilizes category representations through prototype learning.

As shown in Fig. 1, GCN has superior ability to handle irregular data structures compared with CNN. Despite these advantages, several inherent limitations remain in current GNN architectures. Most notably, their heavy reliance on neighborhood aggregation for feature propagation fundamentally restricts their capacity to model long-range dependencies within the data. Although increasing the network depth can theoretically alleviate this issue, it often introduces new challenges, such as the over-smoothing phenomenon, exponential growth in computational complexity, and the undesirable compression of node-level feature representations.

To address the aforementioned issues, we propose a novel graph-based Transformer and convolution fusion network (GTCFN). As illustrated in Fig. 2, the proposed network effectively integrates the global modeling capability of Transformer architectures with the local feature extraction strength of CNNs. Specifically, a 1-D convolution is first employed to compress and transform the spectral features, thereby unifying and simplifying the input representations for both branches. In the S^2CsN branch, a lightweight graph-based Transformer module is designed, which incorporates a lightweight global attention mechanism and a GMRA mechanism. This design not only reduces the computational complexity of the Transformer branch from $\mathcal{O}(M^2)$ to $\mathcal{O}(\mathrm{HKM})$ but also alleviates the over-smoothing problem commonly observed in deep GNNs. Finally, a multiscale spectral–spatial convolutioanal module is developed by introducing convolutional kernels of different sizes to extract complementary spatial–spectral features. The main contributions of this work are summarized as follows.

1) To leverage the complementary strengths of GCNs, Transformers, and CNNs, a dual-branch architecture is proposed, in which a graph transformer subnetwork (GTsN) and a spectral–spatial convolutional subnetwork (S^2CsN) are effectively integrated. This parallel design enables the simultaneous utilization of global contextual information and local discriminative cues, thereby facilitating the comprehensive feature fusion.
2) We design a lightweight kernelized message-passing mechanism that approximates softmax attention via random feature mapping. This reduces the computational complexity of global attention from $\mathcal{O}(M^2)$ to $\mathcal{O}(M)$ while preserving the capacity to model long-range dependencies.
3) To alleviate over-smoothing and improve model generalization, we introduce the GMRA mechanism. By incorporating Gumbel noise into the multihead attention process, this module adds stochasticity to feature aggregation, enhancing robustness without increasing model complexity.
4) We propose a lightweight S^2CsN that integrates multikernel design and depthwise separable convolution within a dual-branch fusion framework, enabling the model to preserve pixel-level features while complementing the global graph-based reasoning.

The structure of this article is arranged as follows. Section II reviews the relevant literature. Section III provides a comprehensive explanation of the proposed algorithm. Experimental settings and results are reported in Section IV, with further discussion and analysis presented in Section V. Finally, Section VI offers the conclusion of this study.

II. Related Works

In this section, we provide a brief overview of three representative models commonly used in HSIC: CNNs, Transformers, and GNNs. This section first introduces the notation and definitions used in this article, as shown in the Nomenclature. Then, it focuses on the basic principles, structural features that describe CNNs, Transformers, and GNNs.

A. Graph Neural Networks

GCNs apply convolution by aggregating information from nearby nodes, allowing effective feature learning on non-Euclidean structures. A graph is denoted by $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where cardinality $|\mathcal{V}| = N$ and $|\mathcal{E}| = E$. The relationships among nodes can be described using an adjacency matrix $\mathbf{A} \in \mathbb{R}^{N\times N}$, where $a(i, j) \neq 0$ indicates the existence of an edge between node i and node j. The corresponding diagonal degree matrix $\mathbf{D}$ is defined as $\mathbf{D}_i = \sum_j \mathbf{A}_{ij}$.

The normalized graph Laplacian matrix is defined as follows:

$$\mathbf{L} = \mathbf{D} - \mathbf{A} = \mathbf{I}_n - \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}} \tag{1}$$

where $\mathbf{I}_n$ is the identity matrix of size $n \times n$. Given an input graph signal x that contains the feature vectors of all nodes in graph $\mathcal{G}$, the graph convolution of x with kernel g in the spectral domain is defined, according to the convolutional theorem, as follows:

$$x *_G g = \mathbf{U}\left(\mathbf{U}^T x \odot \mathbf{U}^T g\right) = \mathbf{U} g \mathbf{U}^T x \tag{2}$$

where $\mathbf{U}$ is the eigenvector matrix of the Laplacian $\mathbf{L}$, $*_G$ is the graph convolution, and $\odot$ denotes the elementwise (Hadamard) product.

To reduce the computational complexity of spectral filtering, Defferrard et al. [44] approximate the spectral convolution kernel g using a truncated expansion of Chebyshev polynomials, resulting in

$$g_\theta(\Lambda) \approx \sum_{k=0}^{K} \theta_k T_k\left(\hat{\Lambda}\right) \tag{3}$$

where K is the order of the Chebyshev polynomial, determining the size of the receptive field, $T_k(\cdot)$ denotes the kth Chebyshev polynomial, and θ_k is the learnable coefficients.

Based on this approximation, the layerwise propagation rule for GCNs is formulated as follows:

$$\mathbf{H}^{l+1} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^l\mathbf{W}^l\right) \tag{4}$$

where $\mathbf{H}^{l+1}$ and $\mathbf{H}^l$ denote the node feature representations at layers $l+1$ and l, respectively, $\mathbf{W}^l$ is the trainable weight

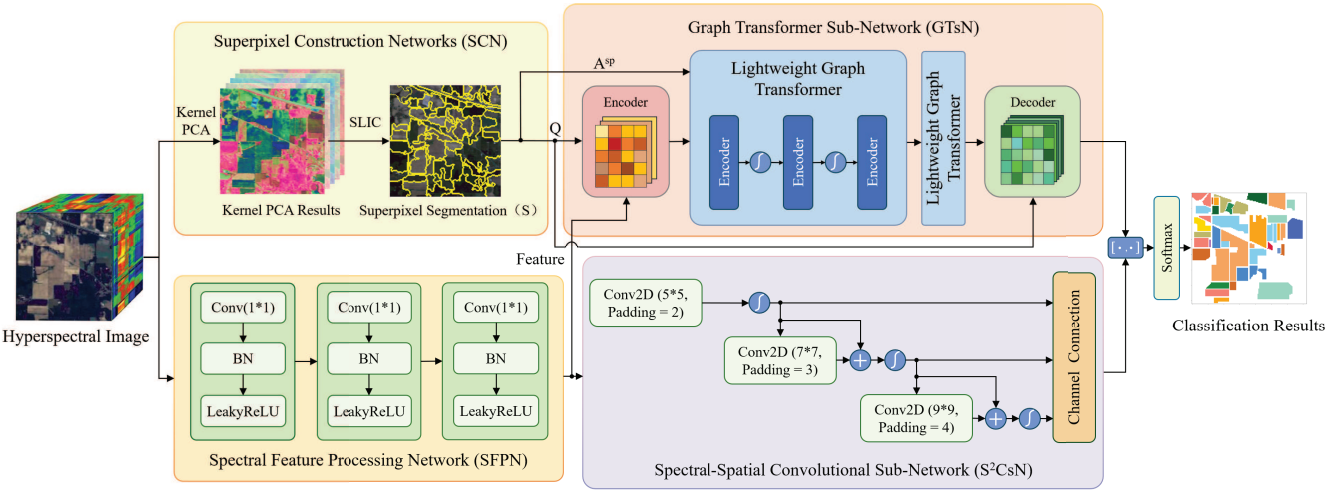


Fig. 2. Overview illustration of the proposed GTCFN. The core of the network consists of four parts: SCNs, SFPN, GTsN, and S^{2}CsN. Outputs from the GTsN and S^{2}CsN branches are fused through a channel connection module and fed into a softmax classifier to produce the final prediction map. $\oplus$ stands for feature fusion, $\mathcal{S}$ is the activation operation, and $[\cdot,\cdot]$ represents the feature concatenation process.

matrix at layer l, and $\sigma(\cdot)$ is a nonlinear activation function. Here, $\tilde{\mathbf{A}} = \mathbf{A}+\mathbf{I}$ is the adjacency matrix with added self-loops, and $\tilde{\mathbf{D}}$ is the corresponding degree matrix.

Based on this, the following simplified node-level update expression can be obtained under the conditions of ignoring self-connections, assuming equal node degrees, and omitting degree normalization:

$$\mathbf{z}_i^{l+1} = \sum_{i=1}^{M} a_{ij} W^l \mathbf{z}_i^l = \sum_{i,(i,j)\in\mathcal{E}} W^l \mathbf{z}_i^l \tag{5}$$

where $\mathbf{z}_i^{l+1}$ denotes the updated feature embedding of node i at layer $l+1$, computed as a weighted sum of transformed neighboring features. a_{ij} is the adjacency matrix and $\mathbf{W}^l$ is the lth layer weight matrix.

B. Transformer

The Transformer architecture overcomes the limitations of traditional recurrent neural networks, which require sequential processing, and CNNs, which struggle to model long-range dependencies. It achieves this by employing a self-attention mechanism that enables the modeling of global dependencies among all positions in a sequence in a fully parallel and nonsequential manner. A Transformer consists of two main components: an encoder and a decoder. Since our task focuses on the image classification, only the encoder module is utilized.

Let the input sequence $\mathbf{X}$ have a length of n, denoted by $\{x_1, x_2, \ldots, x_n\}$, where each token $x_i \in \mathbb{R}^d$ is a d-dimensional feature vector. The encoder maps $\mathbf{X}$ into a continuous representation $\mathbf{Z} = \{z_1, z_2, \ldots, z_n\}$. In the original architecture, the decoder then generates an output sequence $\{y_1, y_2, \ldots, y_n\}$ in an autoregressive manner; however, the decoder is omitted in our classification setting.

To retain positional information, a positional encoding vector p_i is added to each input token x_i to obtain x_i'

$$x_i' = x_i + p_i. \tag{6}$$

In the multihead self-attention mechanism, each vector x_i' is linearly projected into three subspaces to form the query (Q), key (K), and value (V) matrices. Formally, letting $\mathbf{X}' = [x_1', x_2', \ldots, x_n']^\top$, the projections are defined as follows:

$$Q = \mathbf{X}'W^Q, \quad K = \mathbf{X}'W^K, \quad V = \mathbf{X}'W^V \quad . \tag{7}$$

The attention output is computed using the scaled dot-product attention mechanism

$$\mathbf{Z} = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V \tag{8}$$

where d_h is the dimensionality of the query and key vectors. Furthermore, when a multihead self-attention mechanism is adopted, the model is able to simultaneously attend to information from different representation subspaces at multiple positions.

C. Convolutional Neural Networks

CNNs are deep learning models designed for grid-structured data. Hierarchical features are extracted through layered operations using local receptive fields and shared weights. In an HSI of size $H \times W$, each pixel can be directly interpreted as a node, resulting in $H \times W$ distinct nodes across the spatial domain. Let the convolutional kernel (filter) have a size of $k \times k$, with stride s and padding p. Then, the height H_{out} and width W_{out} of the output feature map after a convolution operation can be calculated as follows:

$$\begin{aligned} H_{\text{out}} &= \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1 \\ W_{\text{out}} &= \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1 \end{aligned} \tag{9}$$

where $\lfloor\cdot\rfloor$ denotes the floor operation. Max-pooling and average pooling are commonly used operations that downsample feature maps by selecting the maximum or average value within a small window (e.g., 2×2). Convolution effectively captures textures and edges in fixed-size regions, aiding feature extraction from structured pixels.

III. Methods

In this section, the overall workflow and architecture of the proposed GTCFN framework are elaborated in detail. As illustrated in Fig. 2, GTCFN comprises two complementary subnetworks: S^2CsN and GTsN. Specifically, S^2CsN is designed to extract fine-grained spectral–spatial features at the pixel level, whereas GTsN focuses on modeling contextual dependencies at the superpixel level. Initially, the input HSI is subjected to dimensionality reduction, which compresses redundant spectral information and improves feature discriminability. This step provides a unified and simplified input representation for both branches. In the S^2CsN branch, a multikernel spectral–spatial convolutional module with varying kernel sizes is employed to capture information across different spatial scales, enabling the formation of discriminative pixel-level features. Simultaneously, in the GTsN branch, a kernelized message-passing mechanism is introduced to approximate the self-attention operation via random feature mapping. This strategy significantly reduces the computational complexity of the Transformer branch to $\mathcal{O}(M)$ while preserving its capability for global context modeling. To further mitigate common issues in deep GNNs, such as over-smoothing and overfitting, a Gumbel-based multihead stochastic aggregation (GMSA) module is employed. By injecting controllable stochasticity into the attention mechanism, the model's generalization ability is enhanced under a complexity of $\mathcal{O}$(HKM). Finally, the superpixel-level representations are decoded back to the pixel level and concatenated with the features extracted by the S^2CsN branch. The fused features are subsequently fed into a softmax classifier to produce the final land-cover classification results.

A. Superpixel Construction Networks (SCNs)

In HSIC, a key challenge lies in the fundamental differences in data representation between CNNs and GNNs. CNNs are inherently designed to process regularly structured grid data (e.g., images), whereas GNNs operate on arbitrarily structured graph data. Consequently, the feature representations generated by CNNs and GNNs are often incompatible, making direct integration within a unified framework nontrivial. To address this issue, Liu et al. [40] proposed a structure Transformer that facilitates bidirectional information flow between CNN and GNN modules. This approach effectively preserves the spatial modeling capabilities of CNNs while simultaneously leveraging the relational reasoning strengths of GNNs.

Due to the high spectral dimensionality and the nonlinear distribution characteristics of HSI data, KPCA with a Gaussian kernel is employed to effectively capture underlying nonlinear structures—such as subtle spectral variations between land-cover classes. This choice not only enhances the model's ability to extract discriminative features but also improves robustness to noise and uncertainty in the data distribution. The KPCA transformation using a Gaussian kernel is mathematically expressed as follows:

$$\phi\left(\mathbf{x}, \mathbf{x}'\right) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right) \tag{10}$$

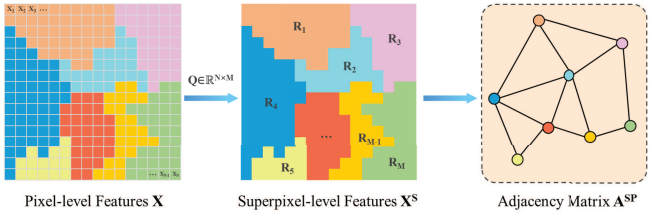


Fig. 3. Flowchart of the SCN module.

where $\mathbf{x}$ and $\mathbf{x}'$ are the spectral vectors, $\|\mathbf{x} - \mathbf{x}'\|^2$ denotes the squared Euclidean distance, and $\gamma = 1/(2\sigma^2)$ is the kernel parameter that controls the degree of nonlinearity.

After dimensionality reduction, we use the simple linear iterative clustering (SLIC) algorithm to segment the HSI from N pixels into M hyperpixels, the overall process is illustrated in Fig. 3. These superpixels represent spatially contiguous regions with similar spectral characteristics, thereby capturing the intrinsic spatial structure of the scene. The number of superpixels is approximately given by $M = (H \times W)/\lambda$, where λ is a user-defined scaling factor that controls the superpixel size. The spectral–spatial characteristics of each superpixel region can be expressed as follows:

$$\mathbf{X}_k^S = \frac{1}{N_k} \sum_{\mathbf{x}_k^i \in R_k} \mathbf{x}_k^i \tag{11}$$

where $\mathbf{X}_k^S$ denotes the kth component of the superpixel-level feature vector $\mathbf{X}^S$ and N_k is the number of pixels contained in the kth superpixel region R_k. Here, $\mathbf{x}_k^i$ represents the feature vector of the ith pixel in R_k, and R_k is the set of all pixels assigned to the kth superpixel.

Building upon the above, we further establish a mapping relationship between pixels and superpixels. According to the association matrix $\mathbf{Q}$ ($\mathbf{Q}_{i,k} = 1$ indicates that the ith pixel belongs to the kth superpixel, and $\mathbf{Q}_{i,k} = 0$ otherwise), the feature representation of all superpixels can be reformulated as follows:

$$\mathbf{X}^S = \left[\mathbf{X}_1^S, \mathbf{X}_2^S, \ldots, \mathbf{X}_M^S\right] = \text{Encode}\left(\mathbf{X}; \mathbf{Q}\right) \tag{12}$$

where Encode$(\mathbf{X}; \mathbf{Q})$ is a graph encoder function that aggregates pixel-level features $\mathbf{X}$ into superpixel-level nodes based on the association matrix $\mathbf{Q}$. Similarly, using the association matrix $\mathbf{Q}$, the graph decoder is further defined as follows:

$$\mathbf{X}^Z = \text{Decode}\left(\mathbf{X}^S; \mathbf{Q}\right) \tag{13}$$

where Decode $(; \mathbf{Q})$ is the graph decoder and $\mathbf{X}^Z$ is the pixel-level feature after back-mapping the graph node features back into image space.

To better leverage geometric information for modeling the physical structure of hyperspectral scenes, we encode node connectivity using an adjacency matrix $\mathbf{A}^{\text{sp}} \in \mathbb{R}^{k \times k}$, where $\mathbf{A}_{i,j}^{\text{sp}}$ represents the connectivity between nodes i and j, which a value of 1 indicates that the corresponding superpixel regions R_i and R_j are adjacent (i.e., they share a boundary). This matrix-based formulation provides an effective representation of the graph's topological structure, enabling efficient computation of geometric relationships through matrix operations.

The adjacency matrix $\mathbf{A}^{\text{sp}}$ serves as the foundation for subsequent graph convolution operations, allowing the network to explicitly incorporate geometric priors derived from the physical layout of the scene while maintaining the computational efficiency. The neighborhood matrix is formally defined as follows:

$$\mathbf{A}_{i,j}^{\text{sp}} = \begin{cases} 1, & \text{if } R_i \text{ and } R_j \text{ are adjacent} \\ 0, & \text{otherwise.} \end{cases} \tag{14}$$

In summary, through the denoising and graph construction processes, the redundant spectral information across hyperspectral bands is effectively reduced, while the noise interference is also mitigated to a certain extent. A bidirectional mapping mechanism is established between pixel-level features and superpixel-level nodes, forming a crucial foundation for collaborative processing in the subsequent dual-branch fusion network. This approach not only aggregates a large number of pixels into a compact set of superpixel nodes—significantly reducing the computational complexity of graph convolutional operations—but also introduces a learnable association matrix $\mathbf{Q}$. This matrix adaptively refines pixel-to-superpixel assignments based on both spectral features and spatial distributions, enabling the network to accurately delineate class boundaries and enhance feature discriminability in an end-to-end manner. Furthermore, the adjacency matrix $\mathbf{A}^{\text{sp}}$, which encodes the neighborhood relationships among superpixels, serves as a geometric prior for the lightweight global attention module. This facilitates improved modeling of local spatial consistency and topological structures, thereby enhancing the network's contextual representation and overall classification performance.

B. Spectral Feature Processing Network (SFPN)

Raw HSIs often contain substantial noise and redundant information, which hinders the ability of deep learning models to effectively learn discriminative spectral features. To address this problem, we propose the SFPN as an initial data processing module, which helps to improve the quality of spectral feature extraction and generate more reliable input representations for subsequent GTsN and S^2CsN. Specifically, SFPN is a lightweight and efficient processing module that integrates progressive spectral compression with deep feature learning, enabling effective modeling of noisy and redundant spectral bands. The output feature map $\mathbf{X}^l$ of the lth spectral convolutional layer is computed as follows:

$$\mathbf{X}_m^l(x,y) = \sigma\left(\sum_k \mathbf{W}_{m,k}^l(x,y) \cdot \tilde{\mathbf{X}}_k^{l-1}(x,y) + B_{m,k}^l(x,y)\right) \tag{15}$$

where $\mathbf{X}_m^l(x,y)$ denotes the output feature at spatial position (x,y) in the mth channel of the lth layer and $\mathbf{X}_k^{l-1}(x,y)$ represents the input feature at the same position in the kth channel of the $(l-1)$th layer. Also, $\tilde{\mathbf{X}}_k^{l-1}(x,y) = \text{BN}(\mathbf{X}_k^{l-1}(x,y))$, $\text{BN}(\cdot)$ denotes the batch normalization operation, which helps to reduce sensitivity to parameter initialization and accelerates network convergence. $\mathbf{W}_{m,k}^l(x,y)$ is the weight of a 1×1 convolutional kernel connecting the kth input channel to the mth output channel, and $B_{m,k}^l(x,y)$ is the corresponding bias term. $\sigma(\cdot)$ is the nonlinear activation function [e.g., rectified linear unit (ReLU)].

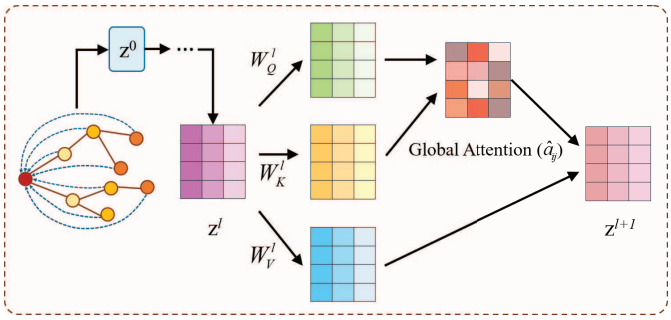


Fig. 4. Schematic of the lightweight global attention mechanism.

By stacking spectral convolutional layers with batch normalization, SFPN not only suppresses noise interference in HSIs and enhances the discriminative power of spectral features for subsequent classification tasks but also maintains computational efficiency by performing convolution operations solely along the channel dimension. This design significantly reduces computational overhead and improves the overall robustness of the model.

C. Graph Transformer Subnetwork

1) Lightweight Global Attention Mechanism: Traditional GNNs are inherently limited in their ability to capture long-range dependencies, as they typically operate only on local neighborhood information. To address this limitation, Transformer architectures have been introduced into graph learning to better capture global contextual relationships [45]. However, standard Transformers pose considerable computational challenges when applied to graphs. Specifically, the self-attention mechanism requires dense pairwise computations between all nodes, which becomes inefficient due to the nonsparse nature of the attention matrix. Moreover, Transformers are inherently sequence-based and thus struggle to model the non-Euclidean structures that characterize graph-structured data. To overcome these issues, Wu et al. [46] proposed NodeFormer, an innovative adaptation that transplants the global attention mechanism of Transformers into the graph domain.

In this section, we will focus on constructing a Transformer-based architecture on the graph for global feature extraction. First, by incorporating the idea of Transformer any two two nodes connected to GNN, we can obtain the information update model, as shown in Fig. 4, the specific formula is given by (16) updated from (5):

$$\mathbf{z}_i^{l+1} = \sum_{j=1}^{M} \hat{a}_{ij}^l \cdot \left(W_V^l \mathbf{z}_j^l\right) \tag{16}$$

$$\hat{a}_{ij}^l = \frac{\exp\left(\left(W_Q^l \mathbf{z}_i^l\right)^\top \left(W_K^l \mathbf{z}_j^l\right)\right)}{\sum_{w=1}^{M} \exp\left(\left(W_Q^l \mathbf{z}_i^l\right)^\top \left(W_K^l \mathbf{z}_w^l\right)\right)} \tag{17}$$

where the attention coefficient $\hat{a}_{ij}^l$ quantifies the similarity between nodes i and j in the projected query and key spaces.

W_Q^l, W_K^l, and W_V^l are the learnable projection matrices in the lth layer, node j is the neighboring node of node i, and w is all nodes.

Equations (16) and (17) are based on the conventional Transformer, in which the interactions between all node pairs are required to be computed individually, leading to a computational complexity of $\mathcal{O}(M^2)$. When applied to HSIs, where M is typically large, this quadratic complexity imposes a substantial computational burden, making it difficult to scale the model to large-scale scenarios. To address this limitation, we introduce an efficient approximation method. First, a positive-definite kernel function $\kappa(\cdot,\cdot)$ is employed to replace the exponential operation in (17), resulting in the following revised formulation:

$$\mathbf{z}_i^{l+1} = \sum_{j=1}^{M} \frac{\kappa\left(\mathbf{q}_i, \mathbf{k}_j\right)}{\sum_{w=1}^{M} \kappa\left(\mathbf{q}_i, \mathbf{k}_w\right)} \cdot \mathbf{v}_j \tag{18}$$

where $\mathbf{q}_i = W_Q^l \mathbf{z}_i^l$, $\mathbf{k}_j = W_K^l \mathbf{z}_j^l$, and $\mathbf{v}_j = W_V^l \mathbf{z}_j^l$, $\kappa(\cdot,\cdot)$ is a positive-definite kernel that measures pairwise similarity between nodes (i, j), replacing the dot-product attention in conventional Transformers. This allows the model to implicitly map features into a higher dimensional space, thereby capturing more complex nonlinear relationships. Based on this, the positive-definite kernel function $\kappa(\cdot,\cdot)$ is further subjected to random feature mapping according to Mercer's theorem

$$\kappa(x, y) = \langle \Phi(x), \Phi(y) \rangle_{\mathcal{H}} \approx \phi(x)^{\top} \phi(y) \tag{19}$$

where $\Phi(x)$ projects the input data x into a high-dimensional Hilbert space $\mathcal{H}$ and $\langle\cdot\rangle_{\mathcal{H}}$ represents the inner product operation within that space. Meanwhile, $\phi(\cdot)$ is a random feature mapping function that projects the input into a lower dimensional Euclidean space, allowing for efficient approximation of the kernel function. Various types of random feature mappings are available, including those corresponding to Gaussian kernels (as used in the aforementioned KPCA), polynomial kernels, and Laplacian kernels. Considering the high dimensionality and complexity of hyperspectral data, we adopt positive random features (PRFs), which provide a favorable balance between approximation accuracy and computational efficiency. With PRF-based mapping, the kernel attention formulation in (18) can be further simplified as follows:

$$\mathbf{z}_i^{l+1} = \frac{\phi(\mathbf{q}_i)^{\top} \sum_{j=1}^{M} \phi\left(\mathbf{k}_j\right) \cdot \mathbf{v}_j^{\top}}{\phi(\mathbf{q}_i)^{\top} \sum_{w=1}^{M} \phi(\mathbf{k}_w)}. \tag{20}$$

In (20), the summation terms in both the numerator and denominator are independent of the node i, and, thus, need to be computed only once for all nodes. In summary, the computational complexity of the method is linear by decomposing the attention weights into inner products of random features, requiring only $\mathcal{O}(M)$ (specifically, $\mathcal{O}(M) = \mathcal{O}(M) + M \cdot \mathcal{O}(1)$). Compared with the standard Transformer attention that requires $\mathcal{O}(M^2)$ operations to compute all pairwise similarities, this optimization allows the global attention mechanism to be implemented efficiently on large-scale graphs, while greatly reducing the overall computational burden and speeding up the classification process.

Algorithm 1 Pseudocode Chart for GMRA

Require: Node features $\mathbf{Z}^{(0)} = \mathbf{X}^S$, adjacency matrix $\mathbf{A}$

for $l = 0$ to $L - 1$ **do**
  $\mathbf{Q}^l \leftarrow W_Q^l \mathbf{Z}^l$, $\mathbf{K}^l \leftarrow W_K^l \mathbf{Z}^l$, $\mathbf{V}^l \leftarrow W_V^l \mathbf{Z}^l$
  for $h = 1$ to H **do**
    $\mathbf{Q}_h \leftarrow \mathbf{Q}^l[:, d_h]$, $\mathbf{K}_h \leftarrow \mathbf{K}^l[:, d_h]$, $\mathbf{V}_h \leftarrow \mathbf{V}^l[:, d_h]$
    for $k = 1$ to K **do**
      Sample $g_j^k \sim \text{Gumbel}(0, 1)$
      $G_k \leftarrow \{e^{g_j^k/\tau}\}_{j=1}^{M}$
      $\tilde{G}_k \leftarrow \text{repeat}(\text{unsqueeze}(G_k, 1), 1, d_h)$
      $\tilde{\mathbf{K}}^{h,k} \leftarrow \tilde{G}_k \odot \phi(\mathbf{K}_h / \sqrt{\tau})$
      $\tilde{\mathbf{Q}}^{h,k} \leftarrow \phi(\mathbf{Q}_h / \sqrt{\tau})$
      $\mathbf{U}^{h,k} \leftarrow (\tilde{\mathbf{K}}^{h,k})^{\top} \mathbf{V}_h$
      $\mathbf{N}^{h,k} \leftarrow (\tilde{\mathbf{K}}^{h,k})^{\top} \mathbf{1}_{M \times 1}$
      $\mathbf{z}_g^{h,k} \leftarrow \frac{\tilde{\mathbf{Q}}^{h,k} \cdot \mathbf{U}^{h,k}}{\tilde{\mathbf{Q}}^{h,k} \cdot \mathbf{N}^{h,k}}$
    end for
    $\mathbf{z}_g^h \leftarrow \frac{1}{K} \sum_{k=1}^{K} \mathbf{z}_g^{h,k}$
  end for
  $\mathbf{Z}^{(l+1)} \leftarrow W_O(\mathbf{z}_g^1 \| \cdots \| \mathbf{z}_g^H)$
end for

Ensure: Predict node labels $\hat{\mathbf{Y}}$

2) GMRA Mechanism: An analysis of (20) reveals that the denominator represents a normalization term composed of the summation over all nodes, which serves to constrain the scale of attention. As the number of nodes M increases, this denominator grows accordingly, leading to a significant reduction in attention weights. Consequently, the output tends to approach a zero vector, resulting in an over-normalization issue that severely impedes backpropagation and model optimization. To address these limitations, a novel differentiable graph structure learning module, termed the GMRA mechanism, is proposed. This mechanism first leverages the structure of the multihead attention architecture, wherein each subspace operates on a distinct subset of feature dimensions. This design effectively reduces the normalization burden within individual attention heads. Subsequently, a stochastic sampling strategy is employed to selectively attend to dominant edges, thereby reducing the scale of the denominator, mitigating normalization imbalance, and preserving essential structural information. The overall architecture of the GMRA module is illustrated in Fig. 5. Meanwhile, Algorithm 1 shows the data logic of the GMRA, which unsqueeze (1) reshapes G_k from (N) to $(N, 1)$, and repeat$(1, d_h)$ replicates it along the column dimension.

Specifically, we employ a multihead attention framework. For each attention head $h \in \{1, \ldots, H\}$, we compute the corresponding query matrix Q_h, key matrix K_h, and value matrix V_h by projecting the input features into distinct subspaces. The update formula for each node can be expressed as follows:

$$\mathbf{z}_m^{l+1} = W_O\left(\|_{h=1}^{H} \mathbf{z}_i^{h,l+1}\right) \tag{21}$$

where $\mathbf{z}_m^{l+1}$ denotes the node updated feature embedding under the multihead attention mechanism, the symbol $\|$ represents the concatenation operation across all H attention heads. By employing the multihead attention mechanism,

each attention head independently captures different types of nodewise similarity—such as spectral, spatial, or textural dependencies—thereby enriching the expressive power of the learned representation.

Moreover, we incorporate Gumbel-Softmax reparameterization to enable efficient and differentiable sparse graph structure learning. To ensure differentiability, we replace discrete sampling with the Gumbel-Softmax approximation, allowing gradient-based optimization of the sparsification process. The improved single-step Gumbel-Softmax aggregation can be formally expressed as follows:

$$\mathbf{z}_g^{l+1} = \frac{\phi\left(\frac{\mathbf{q}_i}{\sqrt{\tau}}\right)^\top \sum_{j=1}^{M} e^{g_j/\tau}\phi\left(\frac{\mathbf{k}_j}{\sqrt{\tau}}\right)}{\phi\left(\frac{\mathbf{q}_i}{\sqrt{\tau}}\right)^\top \sum_{w=1}^{M} e^{g_w/\tau}\phi\left(\frac{\mathbf{k}_w}{\sqrt{\tau}}\right)} \cdot \mathbf{v}_j^\top \tag{22}$$

where $\mathbf{z}_g^{l+1}$ denotes the node updated feature embedding under the Gumbel-Softmax aggregation mechanism and g_j is sampled as independently and identically distributed Gumbel noise. To improve the training stability, this sampling procedure can be independently repeated K times for each node, and the output is obtained by averaging the K aggregation results. The final output of GMRA combines $\mathbf{z}_m^{l+1}$ and $\mathbf{z}_g^{l+1}$, which is expressed as follows:

$$\mathbf{z}_i^{l+1} = W_O\left(\Big\|_{h=1}^{H}\left[\frac{1}{K}\sum_{k=1}^{K}\left(\mathbf{z}_g^{h,k,l+1}\right)\right]\right) \tag{23}$$

$$\mathbf{z}_g^{h,k,l+1} = \frac{\phi\left(\frac{\mathbf{q}_i^h}{\sqrt{\tau}}\right)^\top \sum_{j=1}^{M} e^{g_j^k/\tau}\cdot\phi\left(\frac{\mathbf{k}_j^h}{\sqrt{\tau}}\right)}{\phi\left(\frac{\mathbf{q}_i^h}{\sqrt{\tau}}\right)^\top \sum_{w=1}^{M} e^{g_w^k/\tau}\cdot\phi\left(\frac{\mathbf{k}_w^h}{\sqrt{\tau}}\right)} \cdot \mathbf{v}_j^{h\top} \tag{24}$$

where $\mathbf{z}_g^{h,k,l+1}$ denotes the updated feature embedding of the ith node at layer $l+1$, resulting from the kth Gumbel-Softmax aggregation under the hth attention head. By incorporating Gumbel-Softmax sampling, our model selectively attends to edges with higher attention scores—i.e., the most relevant neighbor nodes—thus effectively avoiding information dilution caused by less informative or irrelevant connections. In addition, the injected Gumbel noise introduces stochastic perturbations that act as an implicit regularizer, similar to dropout. This mechanism prevents the model from overfitting to specific neighbor nodes while maintaining the capacity to learn adaptive and meaningful graph structures.

Through the aforementioned design, the proposed GMRA module offers three notable advantages. First, it effectively models multimodal relationships, including spectral similarity, spatial proximity, and textural correlation. Second, it adaptively filters out noisy or irrelevant edges, thereby enhancing model robustness and generalization. Finally, it enables an end-to-end differentiable architecture while maintaining a favorable tradeoff between expressiveness and computational efficiency, with a controlled model complexity of $\mathcal{O}(\mathrm{HKM})$, where H is the number of attention heads and K is the number of Gumbel samples.

D. Spectral–Spatial Convolutional Subnetworks

While the previously introduced graph-based Transformer network effectively handles non-Euclidean graph structures, it

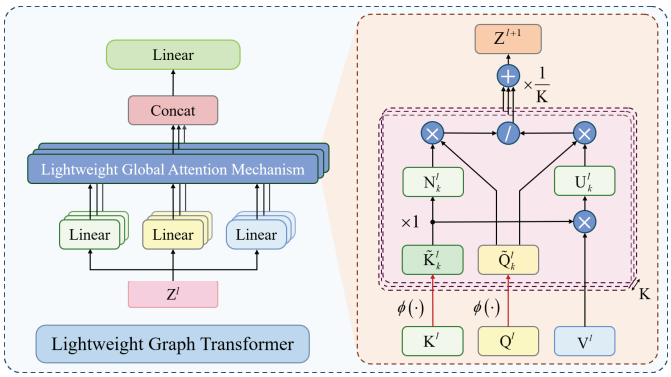


Fig. 5. Schematic of GMRA. (Left) Multihead attention submodule. (Right) Gumbel-Softmax sampling submodule.

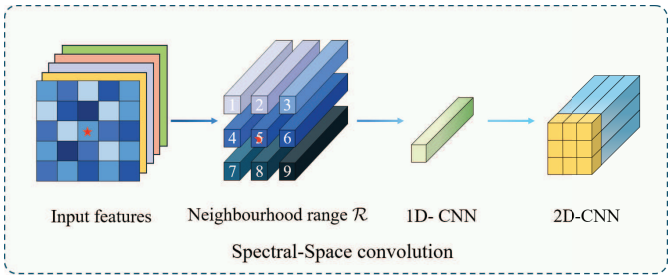


Fig. 6. Illustration of the spectral–spatial convolutional module.

encounters two inherent limitations when applied to regular pixel grids. First, it cannot directly process grid-structured data such as image tensors. Second, its performance is highly dependent on the quality of the predefined graph structure. Moreover, feature aggregation at the superpixel-node level often results in the loss of fine-grained spatial details, which degrades performance in tasks such as small-object classification. To address these issues, we introduce a dual-branch fusion framework that integrates the GTsN with a S^2CsN. This hybrid architecture is specifically designed to preserve pixel-level information from HSIs while retaining the advantages of graph-based representation and global context modeling.

Unlike conventional 3-D convolutions that jointly process spectral and spatial dimensions, the S^2CsN module decouples this operation into successive 1-D and 2-D convolutional steps. The process begins with a 1-D spectral convolution to capture cross-band dependencies, followed by a 2-D spatial convolution to aggregate the local neighborhood information. This decomposition significantly reduces the number of parameters while operating directly on the regular pixel grid, thereby complementing the superpixel-based graph representation and preserving fine-grained features in small or heterogeneous regions. As shown in Fig. 6, the output feature map $\mathbf{T}^l$ of the lth spectral–spatial convolutional layer is computed as follows:

$$\mathbf{T}^l(p_0) = \sigma\left(\sum_{p_n\in\mathcal{R}_l} k^l(p_n)\cdot\sigma\left(\mathbf{W}^l\cdot\tilde{\mathbf{T}}^{l-1}(p_n+p_0-p_l)\right)+B^l\right) \tag{25}$$

where $\mathbf{T}^l$ is the output feature at layer l and $\tilde{\mathbf{T}}^{l-1}$ is the input feature map of the previous layer after batch normalization.

TABLE I
CATEGORYWISE SAMPLE DISTRIBUTION IN THE IP DATASET

No.	Class	Train	Val.	Test
1	Alfalfa	15	1	30
2	Corn-notill	30	1	1397
3	Corn-mintill	30	3	797
4	Corn	30	2	205
5	Pasture	30	3	450
6	Trees/Grass	30	3	697
7	Pasture-mowed	15	1	12
8	Hay-windrowed	30	1	447
9	Oats	15	1	4
10	Soybeans-notill	30	1	941
11	Soybeans-mintill	30	3	2422
12	Soybeans-cleantill	30	2	561
13	Wheat	30	2	173
14	Woods	30	3	1232
15	Residential-Grass-Mixed	30	1	355
16	Stone–Steel Towers	30	2	61
Total		**435**	**30**	**9514**

TABLE II
CATEGORYWISE SAMPLE DISTRIBUTION IN THE PU DATASET

No.	Class	Train	Val.	Test
1	Asphalt	30	4	6597
2	Meadows	30	12	18607
3	Gravel	30	3	2066
4	Trees/Grass	30	1	3033
5	Metal Sheets	30	4	1311
6	Bare Soil	30	2	4997
7	Bitumen	30	2	1298
8	Bricks	30	1	3651
9	Shadows	30	1	916
Total		**270**	**30**	**42476**

TABLE III
CATEGORYWISE SAMPLE DISTRIBUTION IN THE SA DATASET

No.	Class	Train	Val.	Test
1	Broccoli_green_weeds_1	30	2	1977
2	Broccoli_green_weeds_2	30	2	3694
3	Fallow	30	1	1945
4	Fallow_rough_plow	30	2	1362
5	Fallow_smooth	30	1	2647
6	Stubble	30	3	3926
7	Celery	30	2	3547
8	Grapes_untrained	30	3	11238
9	Soil_vineyard_develop	30	3	6170
10	Corn_senesced_green_weeds	30	1	3247
11	Lettuce_romaine_4wk	30	1	1037
12	Lettuce_romaine_5wk	30	2	1895
13	Lettuce_romaine_6wk	30	1	885
14	Lettuce_romaine_7wk	30	2	1038
15	Vineyard_untrained	30	3	7235
16	Vineyard_vertical_trellis	30	1	1776
Total		**480**	**30**	**53619**

k^l denotes the convolutional kernel of the lth layer, and p_l indicates the padding size. p_0 refers to the coordinates of the current output position, while p_n enumerates the relative positions of the convolution kernel within the receptive field $\mathcal{R}_l$. $\mathbf{W}^l$ and B^l represent the learnable weight matrix and bias term of the lth convolutional layer, respectively. The term $p_n + p_0 - p_l$ ensures center alignment within the receptive field, guaranteeing that the convolutional operation correctly maps to the padded input position.

TABLE IV
CATEGORYWISE SAMPLE DISTRIBUTION IN THE HONGHU DATASET

No.	Class	Train	Val.	Test
1	Red roof	30	1	1977
2	Road	30	2	3694
3	Bare soil	30	1	1945
4	Cotton	30	2	1362
5	Cotton firewood	30	1	2647
6	Rape	30	1	3926
7	Chinese cabbage	30	1	3547
8	Pakchoi	30	3	11238
9	Cabbage	30	2	6170
10	Tuber mustard	30	2	3247
11	Brassica parachinensis	30	1	1037
12	Brassica chinensis	30	2	1895
13	Small Brassica chinensis	30	1	885
14	Lactuca sativa	30	1	1038
15	Celtuce	30	2	7235
16	Film-covered lettuce	30	1	1776
17	Romaine	30	1	1776
18	Carrot	30	1	1776
19	White radish	30	1	1776
20	Garlic sprout	30	1	1776
21	Broad bean	30	1	1776
22	Tree	30	1	1776
Total		**660**	**30**	**386003**

On the one hand, the model incrementally captures local features at different receptive field sizes through three layers of spectral–null convolutional networks, employing convolutional kernels of varying scales (5×5, 7×7, and 9×9). This multiscale design enhances the model's ability to detect both fine- and coarse-grained patterns. On the other hand, since the S^2CsN module first performs spectral dimensionality reduction followed by spatial convolution, it significantly reduces the number of parameters compared to conventional 3-D convolutions. Specifically, the parameter complexity is reduced from $\mathcal{O}(d^2mn)$ to $\mathcal{O}((d^2 + n) \cdot m)$, where d denotes the spatial convolutional kernel size, n is the number of input feature channels, and m is the number of output channels. This reformulation considerably improves the model's training efficiency while maintaining its representational capacity.

IV. EXPERIMENTS AND RESULTS

In this section, we evaluate the effectiveness of the proposed methodology through comprehensive experiments conducted on four widely used hyperspectral datasets: Indian Pines (IP), Pavia University (PU), Houston 2013 [Salinas (SA)], and WHU-Hi-HongHu (HongHu). All classification models were implemented using the PyTorch deep learning framework. The experiments were performed on a workstation equipped with an Intel[1] Xeon[1] Platinum 8151 CPU@3.40 GHz and an NVIDIA Quadro GV100 GPU.

A. Description of Experimental Dataset

The IP dataset was acquired by the Airborne Imaging Spectrometer (AVIRIS) over the IP test site in Indiana, USA. The HSI has a spatial resolution of 145×145 pixels, covering

[1]Registered trademark.

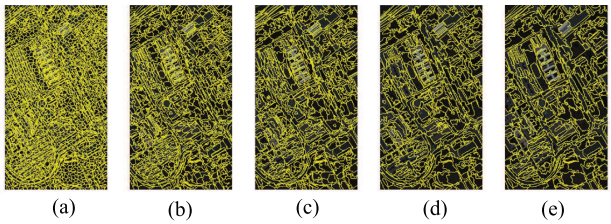


Fig. 7. Visualization of SLIC segmentation results on the PU dataset under different scale parameters λ. (a) λ = 100. (b) λ = 300. (c) λ = 500. (d) λ = 700. (e) λ = 1000.

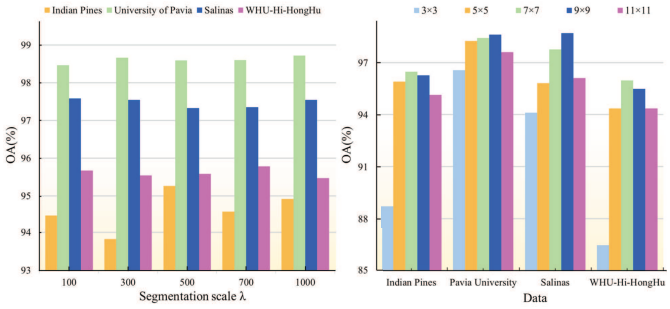


Fig. 8. (Left) OA at different segmentation scales on the four benchmark datasets. (Right) Comparison of OA at different convolutional kernel sizes on the four benchmark datasets.

TABLE V
EFFECT OF GUMBEL SAMPLING ITERATION K

K	1	3	5	7	10	20
IP	95.62	**95.63**	95.57	95.31	94.34	94.32
PU	98.76	98.58	**98.89**	98.50	98.66	95.32
SA	97.40	**97.59**	97.06	97.08	97.54	97.72

an area of approximately 5.76 km^2. The original dataset comprises 224 spectral bands, spanning wavelengths from 0.4 to 2.5 μm. Typically, bands severely affected by water vapor absorption are removed, resulting in 200 effective spectral bands. The scene primarily includes agricultural fields, forests, roads, railways, and buildings, encompassing 16 land-cover categories such as crop types, woodlands, water bodies, and man-made structures. The number of selected samples per class is summarized in Table I.

The PU dataset was acquired by the ROSIS-3 hyperspectral sensor during an airborne campaign over Pavia, Italy. The image has a spatial resolution of 1.3 m and a size of 610 $\times$ 340 pixels. The original dataset consists of 115 spectral bands covering the wavelength range from 430 to 860 nm. In practical applications, 12 bands are typically discarded due to noise, resulting in 103 usable bands. The scene comprises nine land-cover classes, including asphalt, meadows, and bare soil. The distribution of selected samples for each class is summarized in Table II.

The SA dataset was jointly provided by the HSI Analysis Group and the NSF-funded National Center for Airborne Laser Mapping (NCALM) at the University of Houston. This dataset contains 144 spectral bands covering wavelengths from 380 to 1050 nm. The image size is 349 $\times$ 1905 pixels, with a spatial resolution of 2.5 m. The scene includes a diverse range of urban land-cover types, such as healthy grass, stressed grass, artificial turf, and airport runways. The distribution of selected samples across categories is presented in Table III.

The HongHu dataset was acquired using a Headwall Nano-Hyperspec imaging sensor during an airborne survey over Honghu, Hubei, China. The image has a spatial resolution of approximately 0.043 m and a size of 940 $\times$ 475 pixels. The original dataset contains 270 spectral bands covering the wavelength range from 400 to 1000 nm. The scene captures a complex agricultural landscape, featuring a variety of crops such as Chinese cabbage, cabbage, Brassica chinensis, and small Brassica chinensis. The distribution of selected samples across categories is summarized in Table IV.

B. Experimental Settings

1) Evaluation Metrics: In this study, three quantitative evaluation metrics were used to assess classification performance: overall accuracy (OA), average accuracy (AA), and the Kappa coefficient. Specifically, OA measures the proportion of correctly classified samples across all test data, reflecting the overall classification capability of the model. AA is computed as the mean of the per-class accuracies, providing a balanced assessment of performance across different categories. The Kappa coefficient evaluates the agreement between the predicted and ground-truth labels while correcting for chance agreement, thus offering a more robust assessment than OA alone. In addition to these quantitative metrics, we also provide a qualitative evaluation by visualizing the classification results. This helps to intuitively assess the spatial consistency and boundary delineation performance of the proposed model.

2) Comparison Methods and Sample Selection: To comprehensively evaluate the performance of the proposed GTCFN model, we selected ten representative classification methods spanning various technical paradigms and stages of development. The comparison includes one classical machine learning algorithm: SVM [47]; three traditional deep learning models: 3D-CNN [22], HybridSN [24], and SSRN [23]; two advanced Transformer-based approaches: multi-level graph hybrid Transformer (MGHT) [48] and graph structure learning Transformer (NodeFormer) [46]; four state-of-the-art graph-based methods: convolution-enhanced GCN (CEGCN) [40], AMGCFN [41], CNN–GAT-based weighted feature fusion network (WFCG) [49], and graph convolution–Transformer fusion network (GTFN) [29]; and two Mamba-based approaches: spatial–spectral selective structured state-space model (S^{2}Mamba) [50] and integrating mamba and Transformer experts for HSI (HSI-MFormer) [51]. This diverse and extensive set of baseline methods ensures a rigorous comparative framework, enabling a comprehensive assessment of the proposed GTCFN model from multiple technical perspectives.

3) Parameter Settings and Implementation Details: All experiments were conducted on a workstation equipped with an Intel(R) Xeon(R) Platinum 8151 CPU@3.40 GHz and an NVIDIA Quadro GV100 GPU, using the PyTorch deep learning framework. To ensure a fair comparison, we adopted consistent experimental configurations across all methods:

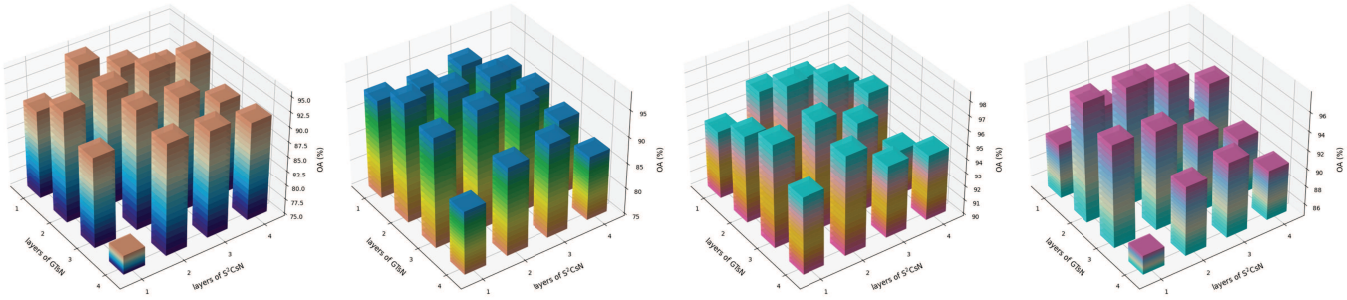


Fig. 9. OA under different network depths l for the GTsN and S^{2}CsN modules across four datasets. (a)–(d) OA curves for the IP, PU, SA, and Honghu datasets, respectively.

TABLE VI

EXPERIMENTAL RESULTS COMPARING DIFFERENT ALGORITHMS ON THE IP DATASET

Class	SVM	3D-CNN	HybridSN	SSRN	MGHT	CEGCN	Nodeformer	AMGCFN	WFCG	GTFN	S^{2}Mamba	HSI-MFormer	GTCFN
1	39.04±1.88	77.83±7.89	93.87±3.42	92.12±6.50	90.63±2.55	93.75±0.54	95.55±0.33	85.71±1.44	93.16±1.95	96.77±2.72	76.37±1.34	**100.00±0.00**	94.44±0.13
2	61.02±6.51	83.61±4.75	82.37±7.72	86.63±8.00	81.34±0.63	84.52±4.62	85.78±9.05	84.07±2.58	87.73±2.32	89.20±1.55	**92.09±1.12**	87.54±0.76	90.61±0.18
3	71.68±3.15	87.59±2.02	92.34±1.68	75.55±9.25	92.00±1.71	83.03±5.98	53.45±1.74	99.52±0.38	96.25±0.31	81.50±3.46	**100.00±0.00**	96.27±0.85	97.92±0.21
4	30.15±7.68	79.67±6.41	92.56±5.24	81.02±4.11	92.45±1.47	84.66±1.24	85.52±0.77	79.20±1.35	86.67±0.16	99.52±0.22	91.28±0.95	**100.00±0.00**	99.66±0.17
5	84.16±4.70	79.31±4.81	97.12±2.45	89.97±4.55	92.70±1.80	91.57±2.05	75.62±2.65	94.58±2.83	85.84±4.57	94.26±1.08	**99.86±0.17**	95.65±0.91	97.58±1.42
6	90.09±1.90	89.08±5.01	96.59±1.96	92.90±3.21	91.91±2.34	99.71±0.13	98.85±1.04	97.03±1.45	92.46±0.15	96.14±3.17	**100.00±0.00**	97.67±0.63	**100.00±0.00**
7	52.65±3.92	76.04±7.64	83.26±6.54	94.30±4.21	88.34±3.29	99.23±0.22	**100.00±0.00**	81.23±1.75	92.38±4.31	92.31±1.28	79.74±1.53	**100.00±0.00**	99.92±0.02
8	89.30±2.37	79.92±7.62	98.12±1.40	95.07±3.76	94.00±0.68	99.55±0.34	97.01±1.67	99.12±0.09	83.24±0.12	98.88±1.21	76.22±1.66	**100.00±0.00**	**100.00±0.00**
9	39.42±8.32	68.65±1.60	95.56±1.78	99.25±0.34	94.44±0.47	98.79±0.41	70.21±3.07	**100.00±0.00**	95.24±3.69	**100.00±0.00**	88.48±1.07	**100.00±0.00**	**100.00±0.00**
10	62.10±4.84	85.65±5.79	90.23±3.44	83.69±7.76	85.92±4.16	84.23±1.62	73.00±3.54	87.68±2.94	92.73±0.33	93.63±2.06	**100.00±0.00**	92.70±0.98	96.53±1.65
11	79.94±4.78	87.16±5.01	84.27±2.83	90.53±4.22	91.62±1.43	96.82±0.66	64.86±3.97	97.62±2.06	**98.47±0.31**	81.65±2.24	96.06±0.92	87.17±1.00	92.36±0.73
12	59.97±1.42	81.36±4.04	93.26±4.53	89.97±4.71	87.70±3.47	89.34±2.95	49.61±5.94	77.28±3.74	86.11±0.18	84.90±1.96	**98.79±0.85**	93.42±0.76	97.39±2.31
13	90.05±3.92	78.91±6.28	99.01±0.86	98.62±1.25	93.22±0.68	98.46±0.75	99.61±0.31	93.26±0.93	99.68±0.09	98.86±0.33	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
14	89.81±3.25	79.73±7.96	95.52±4.05	99.16±0.12	93.33±0.74	98.91±0.97	98.62±0.12	99.67±0.18	84.87±0.13	99.03±0.48	**100.00±0.00**	99.05±0.49	97.30±0.31
15	48.73±1.02	72.79±6.97	85.65±3.28	88.64±5.14	92.78±5.09	99.43±0.37	97.51±4.35	99.73±0.21	92.59±0.69	97.47±1.76	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
16	80.13±1.65	80.01±4.26	95.36±1.85	98.86±1.07	90.55±4.44	98.38±0.55	98.36±1.56	97.80±2.09	89.57±5.68	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**	**100.00±0.00**
OA (%)	73.97±2.83	76.25±3.13	90.11±3.28	87.53±3.32	91.32±1.86	92.51±2.35	72.98±3.83	93.40±3.94	91.74±2.71	90.09±3.26	86.48±1.28	93.33±0.85	**95.62±1.18**
AA (%)	66.82±1.69	76.71±5.52	88.53±2.67	89.12±4.98	91.43±1.99	94.49±1.41	80.22±2.35	92.02±1.19	90.49±1.97	94.01±2.28	93.68±1.02	96.84±0.71	**97.73±0.94**
Kappa	70.29±2.98	76.44±4.88	85.55±3.82	85.76±3.36	90.10±2.13	91.36±2.25	69.19±2.45	92.47±3.26	90.58±2.79	88.73±3.30	84.61±1.41	93.74±0.66	**94.98±1.03**

TABLE VII

EXPERIMENTAL RESULTS COMPARING DIFFERENT ALGORITHMS ON THE PU DATASET

Class	SVM	3D-CNN	HybridSN	SSRN	MGHT	CEGCN	Nodeformer	AMGCFN	WFCG	GTFN	S^{2}Mamba	HSI-MFormer	GTCFN
1	83.82±1.56	84.01±0.65	91.08±0.65	91.08±2.49	97.40±0.14	**99.24±0.18**	86.37±3.56	88.19±4.57	91.61±3.62	84.21±0.73	86.90±0.98	85.89±0.82	97.19±0.12
2	94.75±0.24	83.32±2.75	96.46±4.32	**100.00±0.00**	97.93±0.06	97.50±2.61	97.28±0.23	98.92±0.80	**100.00±0.00**	95.05±0.29	96.17±0.55	97.89±0.45	98.88±1.03
3	56.59±4.22	81.75±1.56	72.35±1.32	80.46±3.26	82.09±0.17	72.67±5.51	72.70±1.01	70.79±5.89	92.16±1.28	78.44±0.79	95.65±0.76	**95.88±0.67**	92.07±1.08
4	74.34±0.96	84.62±2.68	84.92±0.25	86.33±2.69	81.62±0.28	88.44±2.65	77.46±2.32	95.60±2.46	72.08±6.44	89.72±0.83	**98.12±0.34**	95.42±0.74	97.84±0.23
5	72.62±0.65	81.45±4.96	97.32±0.23	94.87±1.23	99.83±0.13	96.16±0.19	98.45±0.65	99.55±0.28	99.69±0.09	98.94±0.46	**100.00±0.00**	99.82±0.09	99.94±0.01
6	71.06±1.63	75.46±9.68	90.63±1.68	82.45±0.45	78.81±0.19	99.56±0.17	31.06±7.53	98.94±0.97	97.96±1.35	99.12±0.23	97.24±0.92	99.93±0.18	**100.00±0.00**
7	68.03±2.58	76.41±1.14	85.51±2.32	95.76±1.20	94.97±0.10	93.76±4.72	97.73±1.20	92.79±1.22	97.02±1.24	97.62±1.89	98.00±0.63	99.97±0.11	**99.82±0.15**
8	81.87±1.96	84.21±0.88	79.28±0.69	90.06±0.96	94.84±0.39	91.09±3.19	94.92±0.96	**99.01±0.30**	88.29±6.24	78.75±0.79	95.04±1.09	84.23±1.03	97.93±1.60
9	68.99±3.20	82.20±0.42	91.34±0.12	81.29±3.69	99.71±0.22	94.91±1.25	99.44±0.45	69.17±7.25	**99.89±0.08**	96.95±1.36	99.67±0.42	97.89±0.53	99.70±0.05
OA (%)	83.73±0.94	82.29±1.97	91.03±0.94	93.05±1.79	93.40±0.81	95.40±2.09	85.08±2.92	94.83±2.98	94.97±2.19	91.50±0.37	95.12±0.48	94.94±0.47	**98.34±0.39**
AA (%)	74.67±1.16	81.49±2.02	87.65±0.82	89.14±1.27	91.91±0.86	92.59±2.12	83.93±2.80	90.35±1.35	93.19±1.57	90.98±0.76	96.31±0.69	95.21±0.32	**98.15±0.34**
Kappa	78.44±0.79	76.54±1.66	88.11±0.96	90.82±1.49	91.25±1.12	93.84±7.28	80.22±2.72	93.17±2.20	93.33±2.76	88.81±0.43	93.58±0.59	93.32±0.62	**97.81±0.64**

each model was trained for 300 epochs with a fixed learning rate of 0.001. For each class, 30 samples were randomly selected for training, while the validation set contained at least one sample per class (30 samples in total). The remaining samples were used as the testing set. All reported results were obtained by averaging the outcomes from ten independent runs, each with different randomly selected training and validation sets. Model optimization was performed using the cross-entropy loss function and the Adam optimizer. For the proposed GTCFN model, the GMRA module was configured with the number of attention heads set to $H = 4$, and the number of Gumbel random samples set to $K = 10$.

C. Hyperparameter Analysis

In this section, we analyze the impact of several key hyperparameters on model performance, including the SLIC segmentation scale (λ), the network depth for each submodule (l), and the convolutional kernel size ($k \times k$) used in the S^{2}CsN module.

1) Effect of Segmentation Scale λ: To determine the optimal segmentation scale λ, we evaluated a range of values: $[100, 300, 500, 700, 1000]$. As shown in Fig. 7 using the PU dataset, segmentation scale directly affects superpixel granularity. Smaller values (e.g., $\lambda = 100$) yield finer boundaries and more superpixels, while larger values (e.g., $\lambda = 1000$) produce coarser regions that suppress noise but lose fine details. The nonlinear relationship between segmentation granularity and classification accuracy is further supported by the OA trends in Fig. 8. PU and SA datasets exhibit clear class boundaries, achieving OAs above 98% and 97%, respectively, demonstrating high robustness. The Honghu dataset achieves peak performance at $\lambda = 700$. In contrast, the IP dataset, due to complex mixed pixels, is highly sensitive to segmentation

TABLE VIII
EXPERIMENTAL RESULTS COMPARING DIFFERENT ALGORITHMS ON THE SA DATASET

Class	SVM	3D-CNN	HybridSN	SSRN	MGHT	CEGCN	Nodeformer	AMGCFN	WFCG	GTFN	S^2Mamba	HSI-MFormer	GTCFN
1	98.94±1.01	97.79±0.13	98.62±0.89	99.51±0.34	99.80±1.01	96.15±1.72	99.50±0.17	95.03±0.88	98.62±0.65	**100.00±0.00**	87.68±1.20	99.98±0.04	98.12±0.25
2	92.33±1.70	94.55±1.72	99.12±0.02	99.54±0.35	99.76±0.13	99.96±0.06	99.74±0.12	**100.00±0.00**	**100.00±0.00**	99.95±0.02	92.42±1.01	**100.00±0.00**	**100.00±0.00**
3	94.75±0.42	92.48±1.21	98.92±0.96	99.63±0.16	99.85±0.17	99.93±0.05	98.36±0.19	97.56±0.79	**100.00±0.00**	**100.00±0.00**	65.31±1.53	**100.00±0.00**	**100.00±0.00**
4	98.91±0.31	98.63±0.14	98.53±0.97	99.53±0.25	99.01±0.44	**100.00±0.00**	84.61±0.66	82.72±3.28	99.92±0.06	99.49±0.09	98.90±0.45	99.61±0.06	99.78±0.18
5	96.27±0.93	90.08±0.73	96.65±0.71	96.04±3.26	98.31±1.07	97.41±1.72	98.59±0.81	97.11±0.71	**99.01±0.54**	93.35±3.56	92.56±1.12	96.82±0.03	93.35±0.15
6	97.49±0.58	96.73±1.27	96.58±0.48	99.48±0.32	99.32±0.06	99.39±0.60	97.74±0.53	97.51±0.10	98.99±0.50	98.77±2.00	99.90±0.33	**99.96±0.03**	99.36±0.48
7	95.22±0.43	97.74±0.14	96.21±2.66	99.35±0.55	96.81±0.37	99.86±0.15	98.69±0.96	98.45±0.64	99.97±0.02	99.89±0.10	50.41±1.26	99.77±0.02	**100.00±0.00**
8	65.59±1.75	72.31±0.59	80.64±2.16	92.24±5.36	88.45±1.48	90.23±1.77	88.32±1.40	**95.13±0.85**	88.36±4.22	85.26±5.78	58.62±1.13	93.57±0.07	85.36±2.29
9	83.96±1.34	95.23±0.86	98.81±0.56	97.79±0.37	98.23±0.58	95.25±0.31	89.26±0.12	**100.00±0.00**	95.59±0.45	99.45±0.37	99.60±0.57	**100.00±0.00**	99.27±0.03
10	81.38±4.38	83.84±0.97	95.90±1.68	95.81±3.78	91.35±1.36	**99.90±0.04**	93.55±0.11	97.18±0.57	97.50±0.58	94.74±5.05	93.97±1.48	96.08±0.06	99.24±0.03
11	94.08±1.59	97.24±0.28	97.94±1.03	99.59±0.24	98.19±1.48	99.43±0.30	97.94±0.81	93.79±0.36	98.78±1.54	**100.00±0.00**	98.84±0.67	97.26±0.02	99.89±0.06
12	98.78±0.79	96.79±0.04	94.65±3.23	99.19±0.02	99.87±0.12	99.88±0.03	96.83±0.49	81.88±2.45	98.86±0.24	99.42±0.05	81.08±1.03	**100.00±0.00**	98.89±0.03
13	96.43±0.39	97.22±0.14	99.20±0.89	98.09±1.33	94.00±2.94	98.83±0.39	97.34±1.33	**100.00±0.00**	99.81±0.05	99.86±0.16	99.44±0.89	98.46±1.07	99.74±0.09
14	98.00±0.54	95.53±1.55	98.98±0.98	99.06±0.38	99.31±0.67	97.83±1.10	95.00±0.75	96.05±0.37	98.72±1.24	98.65±0.99	98.85±0.72	**99.81±0.03**	98.82±0.09
15	75.58±0.45	83.19±3.13	85.85±9.29	79.81±1.77	92.86±3.13	88.79±3.41	73.85±1.65	86.52±1.46	90.12±3.25	87.07±3.35	79.72±1.34	**97.41±0.03**	93.83±0.35
16	95.99±0.37	92.01±1.15	98.73±0.72	98.45±1.02	99.28±0.58	**99.61±0.11**	93.61±0.57	98.88±0.39	95.94±0.78	98.71±1.21	78.90±0.88	98.14±0.03	98.84±0.21
OA (%)	85.18±0.45	88.26±1.39	93.02±4.04	94.67±4.34	95.33±2.11	95.79±0.53	90.68±0.46	95.17±0.06	95.13±1.24	94.32±0.59	86.86±2.81	97.81±0.23	**97.88±0.17**
AA (%)	91.81±1.07	92.65±0.86	96.40±3.14	97.06±2.29	97.15±1.05	97.77±0.37	93.79±0.75	94.77±0.62	97.39±0.77	97.21±0.29	87.01±2.67	98.82±0.10	**98.89±0.12**
Kappa	83.49±0.65	86.93±0.86	92.23±3.39	94.07±2.47	94.80±1.55	95.31±0.59	89.63±0.79	94.62±0.07	94.60±1.56	93.88±0.68	85.46±2.74	97.57±0.24	**97.64±0.11**

TABLE IX
EXPERIMENTAL RESULTS COMPARING DIFFERENT ALGORITHMS ON THE HONGHU DATASET

Class	SVM	3D-CNN	HybridSN	SSRN	MGHT	CEGCN	Nodeformer	AMGCFN	WFCG	GTFN	S^2Mamba	HSI-MFormer	GTCFN
1	94.09±4.99	92.44±2.64	91.65±4.86	93.12±2.98	92.43±2.52	93.49±11.69	90.97±1.99	93.19±1.86	89.77±7.98	93.18±1.41	**95.75±0.92**	95.33±0.30	95.02±1.30
2	87.04±7.14	91.55±1.86	87.61±6.53	85.04±1.58	95.73±1.44	87.29±9.80	95.74±2.25	94.42±1.44	92.49±6.53	91.85±2.48	96.38±0.85	94.97±0.14	**97.88±1.02**
3	93.99±6.13	83.80±2.54	85.14±3.69	96.47±1.43	83.33±1.42	**99.41±0.53**	81.54±4.28	81.04±5.40	89.96±4.82	82.21±6.45	81.74±1.27	85.38±0.17	88.67±2.76
4	74.10±5.97	97.71±1.67	95.26±2.15	91.28±1.19	98.25±0.39	95.95±2.90	96.15±1.77	**99.60±0.09**	95.14±2.30	99.41±0.32	92.51±1.05	93.19±1.21	99.52±0.04
5	80.11±5.56	88.19±1.95	93.81±5.58	86.18±2.43	92.07±2.54	**98.10±1.20**	96.47±1.23	93.70±9.16	96.56±2.62	96.45±1.96	96.51±1.02	97.23±0.62	97.46±1.59
6	76.06±2.55	78.02±5.51	87.10±4.28	91.63±1.80	91.08±2.15	95.98±3.48	92.83±0.62	91.99±4.17	**98.06±1.52**	92.84±1.34	90.68±0.73	93.69±1.16	96.45±1.77
7	91.71±5.31	**98.80±0.73**	83.33±6.26	83.59±7.64	89.83±9.83	78.21±8.83	66.31±3.23	97.43±1.28	91.66±5.75	92.95±4.36	80.12±1.12	85.64±0.85	90.26±8.33
8	72.41±6.14	74.96±6.79	83.20±7.22	63.38±9.82	83.22±1.54	86.16±14.91	76.75±2.59	78.19±11.37	84.21±9.79	75.73±6.55	60.31±0.98	90.86±1.12	**91.88±7.87**
9	98.94±0.02	91.78±1.40	74.60±6.85	92.53±2.49	93.64±1.62	91.85±4.59	92.07±0.76	97.37±1.02	92.53±2.63	82.83±1.31	**99.00±0.49**	97.25±0.45	98.62±0.06
10	62.29±7.60	70.79±3.71	**95.90±1.68**	86.16±4.93	95.41±1.90	87.27±1.51	82.86±1.43	90.84±3.98	86.16±4.93	95.59±1.49	92.20±1.08	93.61±1.45	95.06±4.08
11	74.61±7.50	70.76±11.32	82.75±3.90	72.67±9.84	93.40±9.65	56.43±9.82	93.62±6.72	79.01±7.90	85.67±4.52	67.70±8.74	76.76±1.41	86.09±0.52	**94.81±1.63**
12	76.81±6.34	83.89±9.05	53.09±7.30	80.70±10.68	**90.12±7.17**	89.97±8.77	68.70±3.46	82.54±7.30	86.70±8.64	79.44±9.64	74.11±1.24	87.79±1.38	88.54±1.90
13	70.93±5.06	67.63±8.65	79.95±6.91	86.48±8.10	86.89±0.35	86.11±9.87	74.41±1.37	87.89±0.91	79.48±6.86	69.57±9.56	76.70±0.92	78.28±0.12	**94.78±0.08**
14	87.93±2.16	66.82±6.12	74.65±8.49	89.10±5.94	83.00±1.61	87.76±9.93	83.70±1.35	91.74±1.49	82.10±7.61	83.36±5.21	90.02±0.78	93.19±1.13	**97.09±0.88**
15	77.52±6.09	98.44±0.13	94.43±1.31	83.77±4.30	97.08±1.66	98.45±0.64	97.48±0.15	99.00±0.31	99.77±0.16	99.56±0.42	**100.00±0.00**	**100.00±0.30**	99.36±0.15
16	92.31±2.06	93.89±1.76	84.91±4.82	93.19±2.86	95.64±2.86	93.27±5.49	96.73±0.23	88.24±0.79	93.19±4.53	87.65±5.72	90.96±0.67	98.05±0.82	**99.36±0.02**
17	82.90±5.53	83.08±5.45	92.34±5.10	97.02±0.52	92.59±2.59	94.49±1.27	94.52±0.72	92.19±4.25	91.02±8.28	92.13±0.42	96.61±0.84	96.98±0.96	**97.33±0.24**
18	96.25±1.35	89.79±5.03	91.14±2.43	82.21±1.53	96.52±1.88	98.35±1.01	97.08±0.20	94.19±2.65	92.21±4.30	**99.11±0.21**	96.52±0.83	97.75±0.54	97.16±0.89
19	74.55±6.04	82.20±6.73	92.26±2.46	87.45±5.36	87.04±2.83	93.15±1.74	92.58±5.51	91.23±1.60	94.04±2.83	**97.04±0.58**	94.51±0.71	93.05±1.41	96.77±0.24
20	79.58±5.29	95.62±1.28	89.80±8.08	94.68±1.48	97.08±3.66	94.62±3.36	97.35±0.94	98.75±1.08	94.68±11.85	**99.03±0.53**	97.08±0.66	97.83±1.26	98.71±0.16
21	80.52±8.62	92.01±1.19	90.47±3.13	84.08±5.55	97.73±2.43	86.72±2.63	93.76±2.35	93.47±4.13	91.19±3.67	95.07±2.38	**100.00±0.00**	99.61±0.17	97.06±0.56
22	94.12±1.09	93.57±2.03	95.34±3.81	92.75±2.75	93.46±1.49	98.05±0.10	98.49±0.07	96.77±2.81	94.75±3.80	89.94±0.05	**99.65±0.55**	96.48±0.82	98.92±0.94
OA (%)	78.83±3.40	87.69±4.56	87.36±2.65	89.47±2.18	94.82±0.59	92.30±1.37	89.90±1.81	94.22±1.54	92.54±1.34	92.21±1.26	89.37±0.53	91.67±0.35	**96.69±1.37**
AA (%)	82.67±3.09	83.44±4.13	83.19±2.97	86.98±2.02	93.62±1.97	90.50±1.14	89.10±1.24	91.49±1.27	90.97±1.73	89.07±0.57	89.91±0.61	93.28±0.15	**95.96±1.82**
Kappa	73.36±4.69	84.46±5.67	84.30±3.57	86.37±2.73	93.52±2.37	90.39±2.60	87.24±1.87	92.73±2.65	90.67±2.73	90.23±4.82	86.79±0.57	89.63±0.42	**95.81±1.73**

scale. It reaches its best OA (95.26%) at $\lambda = 500$, while both coarser ($\lambda = 300$ and OA = 93.82%) and finer segmentations degrade performance.

The narrow OA variation across λ values, especially for SA and PU, highlights the robustness of GTCFN to segmentation scale changes. This stability is attributed to two design choices: KPCA preprocessing, which reduces spectral noise and enhances regional features for SLIC clustering; and the graph–Transformer branch, which adaptively models intersuperpixel relationships to mitigate segmentation inconsistencies. Based on these results, $\lambda = 500$ is adopted as the default setting for all datasets, as it consistently yields near-optimal or peak OA.

2) Effect of Convolutional Kernel Size $k \times k$: To investigate the impact of convolutional kernel size on the classification performance of the proposed multiscale convolutional subnetwork, we conducted controlled experiments on four benchmark datasets: IP, PU, SA, and HongHu. In each experiment, the network used a single fixed kernel size selected from the set $\{3 \times 3,\ 5 \times 5,\ 7 \times 7,\ 9 \times 9,\ 11 \times 11\}$, while keeping all other configurations unchanged. The classification results, evaluated by OA, are presented in Fig. 8.

As shown in Fig. 8, the 9×9 convolutional kernels yield the highest OA (98%) on the PU and SA datasets, suggesting that a broader spatial context benefits classification in relatively homogeneous regions. In contrast, the IP and HongHu datasets—characterized by greater spatial heterogeneity and mixed-pixel distributions—perform best with 7×7 kernels. For these datasets, overly large kernels tend to cause over-smoothing, reducing the model's ability to capture local discriminative features. Overall, the best performance is consistently obtained with mid-to-large kernels—specifically 5×5, 7×7, and 9×9—depending on the dataset's spatial characteristics. Accordingly, the proposed multiscale design integrates these kernel sizes to balance local detail preservation and broader feature learning. This enables the S^2CsN module to effectively aggregate complementary spatial features across multiple receptive fields, enhancing adaptability to diverse spatial structures in hyperspectral imagery.

3) Effect of Network Depth l: To evaluate the impact of network depth on the classification performance of the proposed GTCFN, we conducted a series of experiments by varying the number of layers l in its two core subnetworks. The GTsN is responsible for capturing nonlocal and global contextual dependencies through graph-based self-attention mechanisms, while the S^2CsN is designed to extract multiscale local features

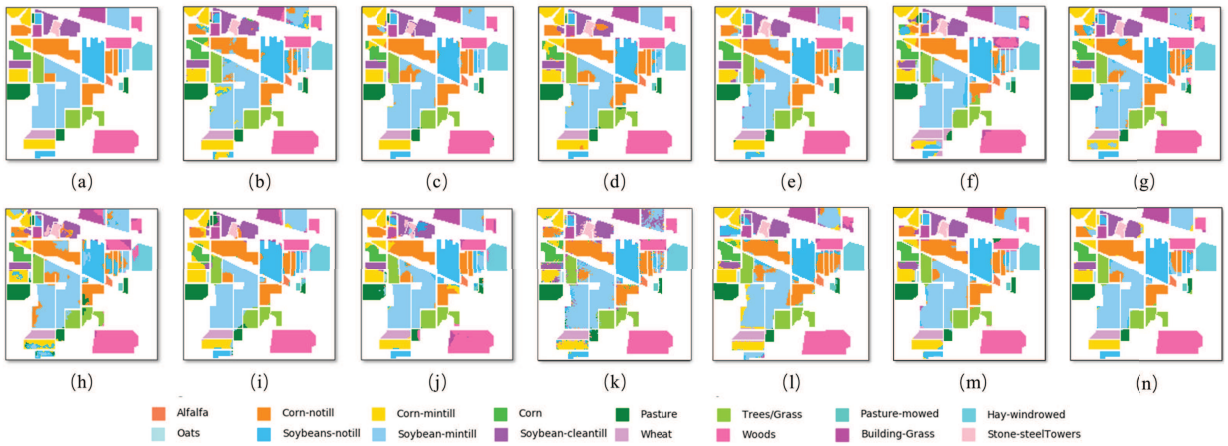


Fig. 10. Experimental results comparing differential algorithms on the IP dataset. (a) Ground truth. (b) SVM (73.97%). (c) 3D-CNN (76.25%). (d) HybridSN (90.11%). (e) SSRN (87.53%). (f) MGHT (91.32%). (g) CEGCN (92.51%). (h) NodeFormer (72.98%). (i) AMGCFN (93.40%). (j) WFCG (91.74%). (k) GTFN (90.09%). (l) S^2Mamba (86.48%). (m) HSI-MFormer (93.33%). (n) GTCFN (95.62%).

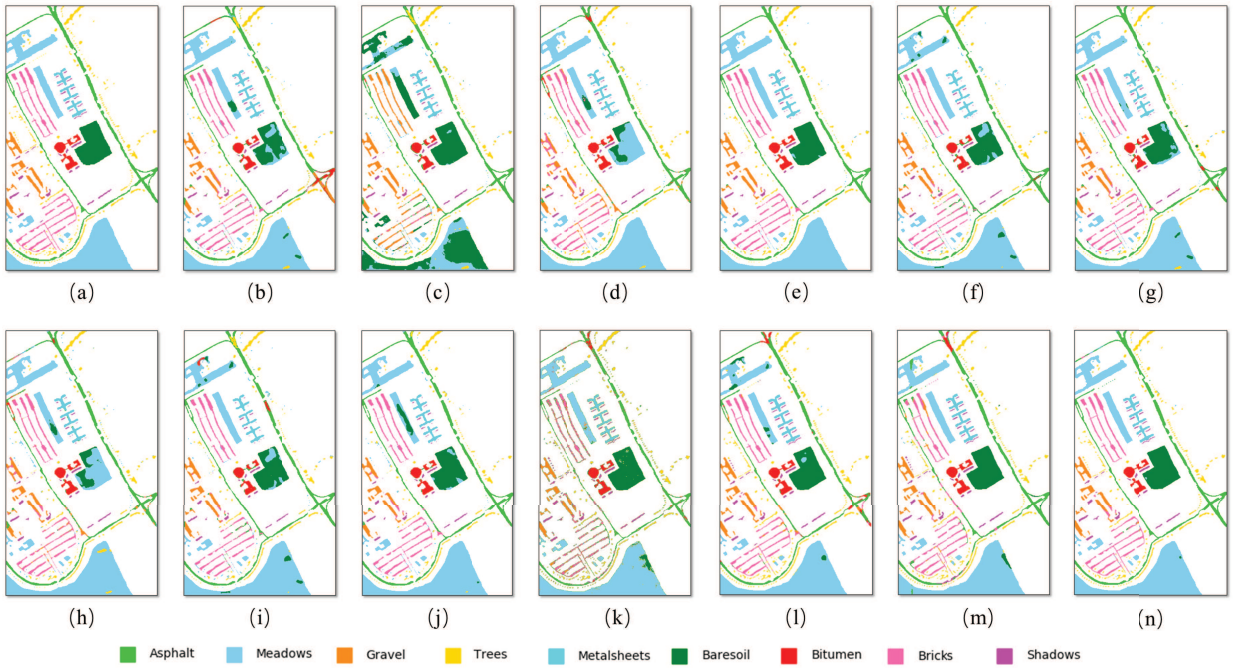


Fig. 11. Experimental results comparing differential algorithms on the PU dataset. (a) Ground truth. (b) SVM (83.73%). (c) 3D-CNN (82.29%). (d) HybridSN (91.03%). (e) SSRN (93.05%). (f) MGHT (93.40%). (g) CEGCN (95.40%). (h) NodeFormer (85.08%). (i) AMGCFN (94.83%). (j) WFCG (94.97%). (k) GTFN (91.50%). (l) S^2Mamba (95.12%). (m) HSI-MFormer (94.94%). (n) GTCFN (98.34%).

through stacked spectral–spatial convolutional operations. In the experiments, the number of layers in each branch was varied from 1 to 4, and the resulting OA was recorded across four benchmark datasets. As shown in Fig. 9, the depth of each subnetwork has an impact on performance.

As shown in Fig. 9, the OA improves significantly with an increasing number of layers across all four datasets. However, beyond a certain depth, the performance either plateaus or slightly declines. The optimal OA is typically achieved when the GTsN contains 2–3 layers and the S^2CsN incorporates 2–3 convolutional layers. Considering the tradeoff between classification performance and computational efficiency, the depth of GTsN is fixed at 2 and that of S^2CsN at 3 in all subsequent experiments, ensuring a balanced architecture between the two branches.

4) Effect of Gumbel Sampling Iterations K: To evaluate the sensitivity of the proposed model to the number of Gumbel sampling iterations K in the GMRA mechanism, we conducted a robustness analysis on the IP, PU, and SA datasets. The value of K was varied within the set $1, 3, 5, 7, 10$, and 20, while all other parameters remained unchanged.

As shown in Table V, the overall performance of the model remains stable across different values of K, indicating that the method is robust to the choice of K. Specifically, on the IP dataset, the classification accuracy consistently remains within a high range of 94.32%–95.63%, peaking at 95.63%

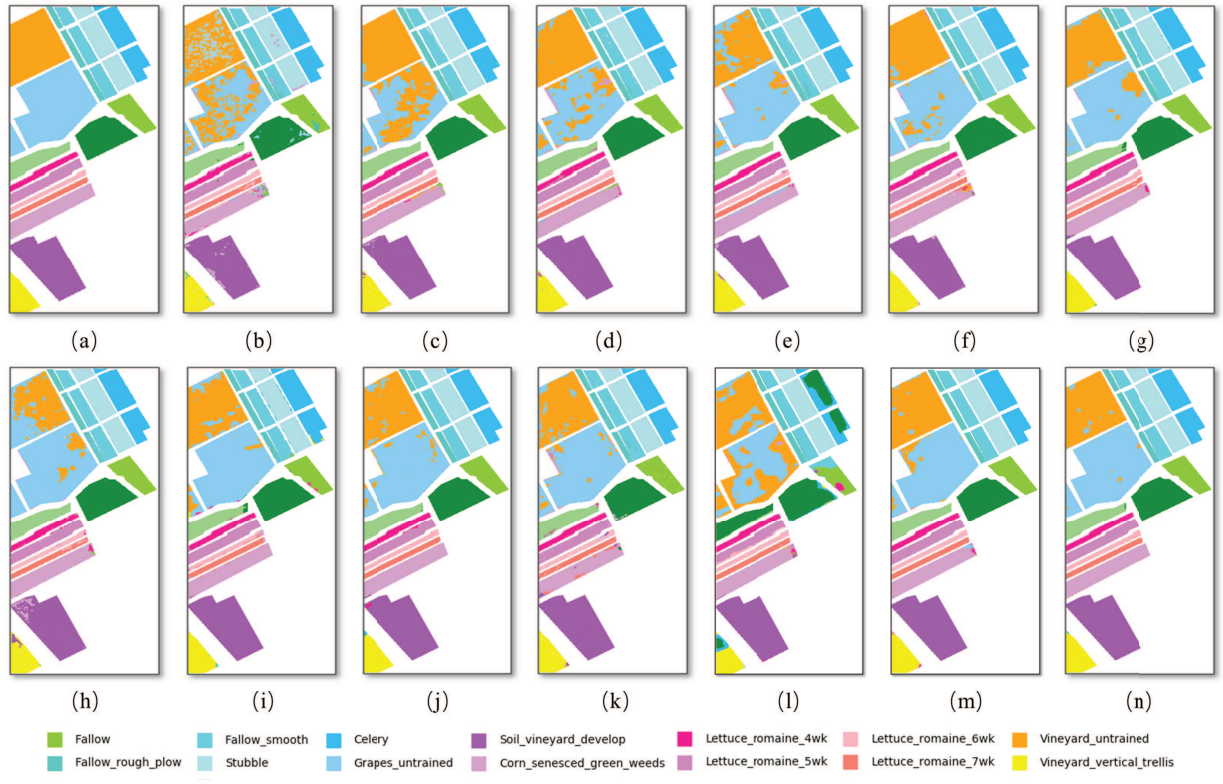


Fig. 12. Experimental results comparing different algorithms on the SA dataset. (a) Ground truth. (b) SVM (85.18%). (c) 3D-CNN (88.26%). (d) HybridSN (93.02%). (e) SSRN (94.67%). (f) MGHT (95.33%). (g) CEGCN (95.79%). (h) NodeFormer (90.68%). (i) AMGCFN (95.17%). (j) WFCG (95.13%). (k) GTFN (94.32%). (l) S^2Mamba (86.86%). (m) HSI-MFormer (97.81%). (n) GTCFN (97.88%).

when $K = 3$, which suggests relative insensitivity to the number of iterations. On the PU and SA datasets, moderate values of K (e.g., $K = 3$ or 5) yield the best performance, indicating that an appropriate level of stochastic sampling helps enhance the feature representation without introducing excessive noise. However, when K increases to 20, a slight performance degradation is observed across all datasets, with accuracies dropping to 94.32% and 95.32% on the IP and PU datasets, respectively. This decline may be attributed to over-smoothing or optimization instability caused by an excessive number of aggregation paths. These results validate our design strategy of using a moderate K to balance diversity and stability. In subsequent experiments, we fix $K = 3$.

D. Classification Results

This section presents a comparative evaluation of the proposed GTCFN model against both traditional machine learning algorithms and advanced deep learning methods.

1) Comparative Experimental Analysis: Tables VI–IX report the quantitative classification results of ten representative algorithms on the IP, PU, SA, and HongHu datasets, including OA, AA, and classwise accuracies across all categories. In the table, the optimal results have been bolded.

On the IP dataset—characterized by high spectral similarity, irregular land parcels, and complex class boundaries—the proposed GTCFN model achieves an OA of 95.62%, an AA of 97.73%, and a Kappa coefficient of 94.98, outperforming all comparison methods. Compared with the recent graph-based models WFCG (OA = 91.74%) and AMGCFN (OA = 93.40%), GTCFN improves classification accuracy by over 2%. In addition, compared with the newly introduced S^2Mamba (OA = 86.48%) and HSI-MFormer (OA = 93.33%), GTCFN achieves an improvement of 9.14% and 2.29% in OA, respectively, further confirming its superiority over the latest state-of-the-art models. Furthermore, it significantly surpasses traditional CNN-based approaches, such as 3D-CNN (OA = 76.25%) and HybridSN (OA = 90.11%), highlighting its superior capacity for modeling complex spectral–spatial relationships. Notably, the performance gains are particularly pronounced in spectrally mixed or underrepresented classes, where conventional methods often struggle. These results demonstrate the effectiveness of the dual-branch architecture in GTCFN: GTsN captures long-range contextual dependencies, while the S^2CsN enhances local feature extraction. Together, they provide complementary strengths that enable the robust classification under challenging intraclass and interclass variability.

On the PU dataset—which features high spatial resolution and strong boundary consistency between classes—the proposed GTCFN model achieves an OA of 98.34%, an AA of 98.15%, and a Kappa coefficient of 97.81. Compared with recent graph-based models such as AMGCFN (OA = 94.83%), WFCG (OA = 94.97%), and GTFN (OA = 91.50%), GTCFN yields OA improvements of 3%–6% and gains of over five points in Kappa. In addition, compared with S^2Mamba (OA = 95.12%) and HSI-MFormer (OA = 94.94%), GTCFN

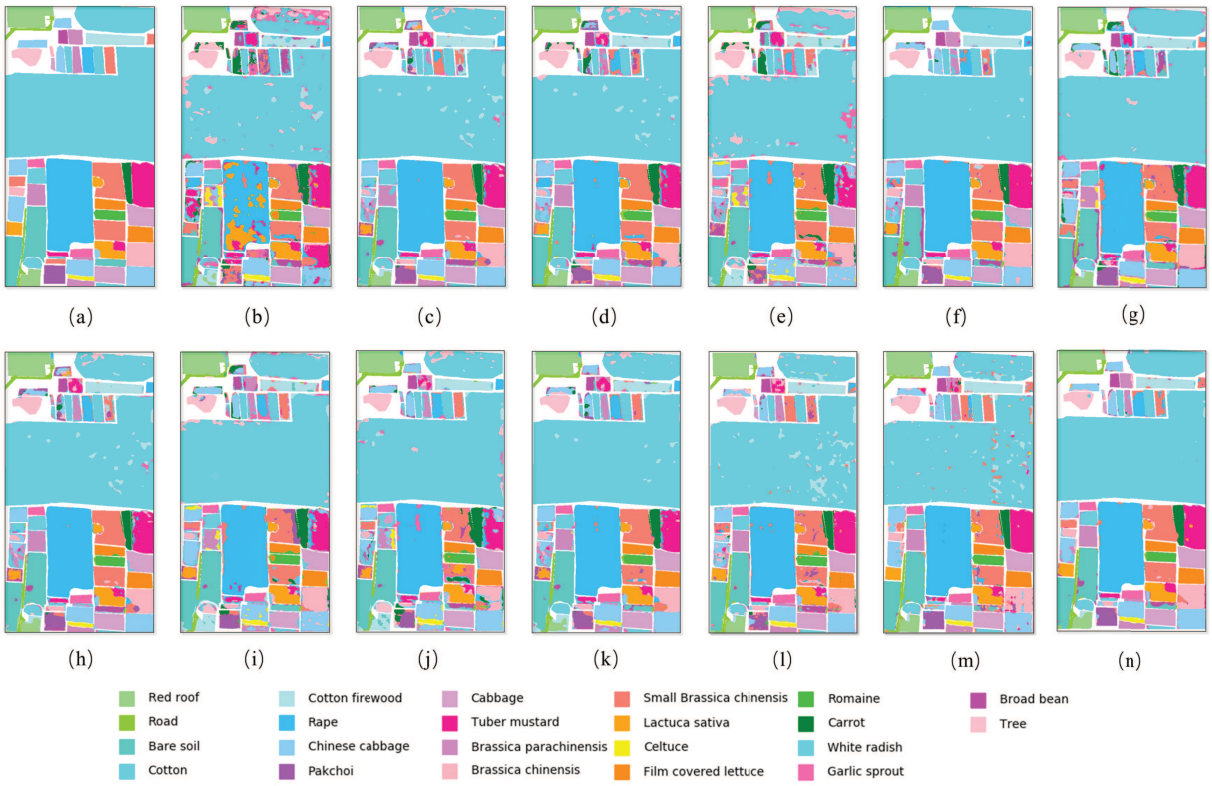


Fig. 13. Experimental results comparing different algorithms on the HongHu dataset. (a) Ground truth. (b) SVM (78.83%). (c) 3D-CNN (87.69%). (d) HybridSN (87.36%). (e) SSRN (89.47%). (f) MGHT (94.82%). (g) CEGCN (92.30%). (h) NodeFormer (89.90%). (i) AMGCFN (94.22%). (j) WFCG (92.54%). (k) GTFN (92.21%). (l) S^2Mamba (89.37%). (m) HSI-MFormer (91.67%). (n) GTCFN (96.69%).

achieves further OA improvements of 3.22% and 3.40%. This significant performance advantage can be attributed to the model's ability to effectively extract and integrate spatial details across multiple receptive fields through the S^2CsN module, while the GTsN branch ensures semantic consistency is preserved across region-level superpixels. The synergistic fusion of local and global information, coupled with KPCA-based spectral feature enhancement, enables GTCFN to deliver robust performance in complex urban scenes, where precise boundary delineation and contextual continuity are essential for accurate classification.

On the SA dataset—which consists of large-scale agricultural fields with regular shapes and relatively low interclass spectral confusion—the proposed GTCFN model achieves an OA of 97.88%, an AA of 98.89%, and a Kappa coefficient of 97.64, once again outperforming all competing methods. In addition, we observe that multiple algorithms in certain categories of the IP, PU, and SA datasets show near-perfect classification results (e.g., 100.00% $\pm$ 0.00). This is not by chance, but rather due to the inherent spectral space properties of these categories, consistent sampling protocols, and the powerful modeling capabilities of our proposed GTCFN and other state-of-the-art comparison methods. However, on the whole, GTCFN still has a clear advantage over the recently proposed S^2Mamba (OA = 86.86%), with an 11.02% improvement in OA over S^2Mamba. Although several advanced models, such as HSI-MFormer (OA = 98.71%), WFCG (OA = 95.13%), and AMGCFN (OA = 95.17%), attain competitive performance, GTCFN consistently achieves higher classwise accuracy and lower variance across categories. This suggests enhanced stability and generalization ability, even under conditions of limited training supervision. Despite the relatively structured and homogeneous nature of the Salinas scene, GTCFN maintains superior classification precision, further validating its adaptability to both complex and regular spatial environments.

On the HongHu dataset—which exhibits the highest spatial and spectral complexity among all benchmark test sets—the proposed GTCFN model achieves the best performance, with an OA of 96.69%, an AA of 95.96%, and a Kappa coefficient of 95.81. Notably, GTCFN also achieves the highest classification accuracy in 12 out of the 22 semantic categories. Compared with strong baselines such as S^2Mamba (OA = 89.37%), HSI-MFormer (OA = 91.67%), WFCG (OA = 92.54%), and CEGCN (OA = 92.30%), GTCFN improves both OA and Kappa by over 4%, and significantly outperforms traditional methods including SSRN, HybridSN, and SVM. GTCFN's strong performance stems from its dual-branch architecture, where a superpixel-based branch ensures spatial coherence and reduces intraclass variance, while the GTsN module captures high-level topological relationships among irregular regions. These innovations collectively enable robust discrimination of fine-grained categories under sparse training conditions and complex spatial–spectral distributions. That said, we acknowledge that in certain complex scenarios—such as class 5 in the SA dataset and classes 7 and 8 in the HongHu

dataset—the model still exhibits minor classification errors, primarily due to high intraclass variability or extreme spectral similarities between adjacent classes.

2) Visual Analysis of Classification Results: This study presents a visual comparison and qualitative evaluation of the classification performance across different methods by displaying the corresponding prediction maps. Figs. 10–13 illustrate the classification results on the four benchmark datasets—IP, PU, SA, and HongHu—highlighting the visual differences in classification quality among competing algorithms.

Fig. 10 illustrates the classification results of various algorithms on the IP dataset. It is evident that traditional approaches such as SVM and 3D-CNN exhibit substantial fragmentation and noise, particularly across large homogeneous regions, resulting in blurred class boundaries and poor spatial consistency. In contrast, models—such as HybridSN, SSRN, and MGHT—show notable improvements in preserving region integrity and delineating clearer boundaries, reflecting stronger spatial-awareness capabilities. Notably, CEGCN and AMGCFN demonstrate superior spatial–spectral joint modeling by effectively recovering the contours of multiple land-cover patches. Notably, the recently proposed Mamba-based models (S^2Mamba and HSI-MFormer) also deliver competitive results with smoother regional consistency. Among all methods, the proposed GTCFN achieves the highest OA (95.62%) and delivers the most visually coherent classification map, with accurate structure reconstruction and continuous, well-aligned boundaries. These visual results further validate GTCFN's robust modeling capacity and superior generalization performance in complex hyperspectral classification tasks.

Fig. 11 presents the classification results of various algorithms on the PU dataset. In contrast, the proposed GTCFN method demonstrates superior spatial–spectral modeling capabilities, effectively recovering the shapes of major land-cover categories such as trees, bare soil, and metal sheets. GTCFN also enhances spatial continuity, yielding compact and coherent classification regions with clear boundaries and minimal noise. These improvements are consistent with the performance trends observed on the IP dataset, further validating the robustness of GTCFN in diverse spatial contexts.

Fig. 12 displays the classification results of different algorithms on the SA HSI dataset. Traditional methods such as SVM and 3D-CNN exhibit severe fragmentation, particularly along class boundaries and in minority class regions (e.g., orange, blue, and green areas), with noticeable "salt-and-pepper" noise degrading overall visual coherence and classification reliability. In contrast, the proposed GTCFN method achieves the highest OA (97.88%) and generates visually clean outputs with minimal noise in large homogeneous areas, while maintaining high precision in complex or detail-dense regions. Compared to other approaches, GTCFN clearly excels in classification consistency, fine-detail preservation, and global spatial smoothness across the entire scene, confirming its strong generalization ability in structured agricultural landscapes.

Fig. 13 shows the comparsion of the classification results of various algorithms on the HongHu dataset, which comprises numerous regularly shaped farmland blocks with clearly delineated object boundaries. As shown in the figure, traditional models struggle with accurate region reconstruction, exhibiting substantial boundary ambiguity and class confusion. Notably, large cotton bodies (cyan) contain significant noise, and the distinction between adjacent farmland plots is often lost. In contrast, the proposed GTCFN method achieves the highest OA (96.69%) and substantially reduces misclassification. It preserves continuous and smooth field boundaries, delivers clean segmentation within cotton regions, and produces a visual output that most closely resembles the ground truth. These results underscore GTCFN's strong capability in joint spatial–spectral representation learning and its effectiveness in suppressing local classification errors in complex agricultural scenes.

TABLE X
RESULTS OF DIFFERENT CONFIGURATIONS FOR A SEGMENTATION MODEL

Data	SFPN	GTsN	S^2CsN	OA(%)	AA(%)	KAPPA
IP	×	√	√	92.34	96.95	91.51
	√	×	√	80.47	89.82	78.01
	√	√	×	41.16	60.55	35.09
	√	√	√	**95.80**	**98.32**	**95.19**
PU	×	√	√	95.45	95.64	94.94
	√	×	√	92.01	93.58	89.45
	√	√	×	55.85	63.10	50.40
	√	√	√	**97.61**	**97.49**	**96.83**
SA	×	√	√	90.76	96.49	89.77
	√	×	√	92.03	94.34	91.45
	√	√	×	61.53	74.45	56.68
	√	√	√	**97.55**	**99.05**	**97.25**

V. DISCUSSION

A. Ablation Analysis

To assess the contribution of each component within the proposed GTCFN architecture, we conducted a series of ablation experiments. The results, as summarized in Table X, report the classification performance in terms of OA, AA, and the Kappa coefficient.

On the IP dataset, both GTsN and S^2CsN are shown to be indispensable. Removing GTsN causes the OA to drop sharply from 95.80% to 80.47%, with the Kappa coefficient decreasing from 95.19 to 78.01, highlighting the critical role of global context modeling through graph-based reasoning. Conversely, removing S^2CsN results in a dramatic OA decline to 41.16%, underscoring the necessity of local spatial and texture modeling for preserving structural details. For the PU dataset, which features more regular and spatially coherent urban scenes, a similar trend is observed: the full-GTCFN model consistently outperforms its ablated variants. On the Salinas dataset, despite its high spatial regularity and interclass separability, the full model still achieves superior performance, reaching 97.55% OA, 99.05% AA, and a Kappa score of 97.25. Removing GTsN reduces OA to 92.03%, while the absence of S^2CsN causes a substantial accuracy drop to 61.53%. These results reaffirm that both the graph-based and convolutional branches contribute significantly, even in structurally simple scenarios.

TABLE XI
COMPARISON OF THE OPERATIONAL EFFICIENCY OF DIFFERENT METHODS

Method	IP		PU		SA		HONGHU		Parameters
	Train (s)	Test (s)	Train (s)	Test (s)	Train (s)	Test (s)	Train (s)	Test (s)	(HongHu)
3D-CNN	87.96	4.82	54.87	10.34	93.43	11.45	139.16	116.51	585570
HybridSN	109.08	3.24	60.81	11.49	115.04	13.56	206.16	230.15	493359
SSRN	105.24	3.39	78.12	10.55	117.72	11.98	154.97	225.26	722398
MGHT	145.72	2.75	183.32	13.98	117.47	10.83	295.48	34.61	965760
CEGCN	45.75	2.35	89.46	10.21	53.27	6.85	216.97	35.12	176244
AMGCFN	86.23	1.14	199.93	14.15	156.25	9.15	574.35	59.92	386703
WFCG	282.34	3.98	589.12	16.45	729.55	11.45	2264.77	68.28	111912
GTFN	121.42	18.35	297.65	49.56	310.13	52.56	1947.97	563.54	282658
S^2Mamba	145.32	9.14	169.83	24.50	319.84	42.15	1498.76	308.16	368040
HSI-MFormer	124.17	2.42	149.65	8.96	144.40	11.98	156.86	149.60	1749216
GTCFN	**8.07**	**0.48**	**38.09**	**4.54**	**25.42**	**2.75**	**93.44**	**13.52**	**234708**

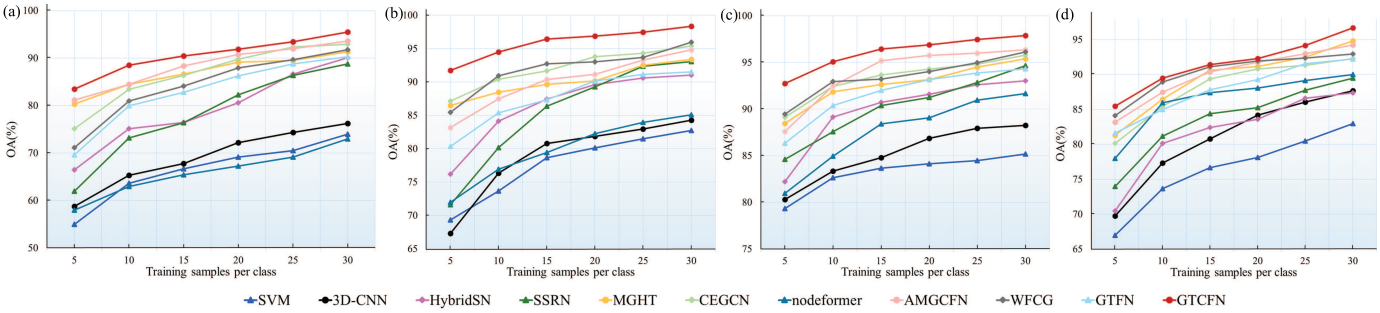


Fig. 14. OA for different proportions of training samples on the four datasets. (a)–(d) OA curves for the IP, PU, SA, and Honghu datasets, respectively.

These findings confirm that GTsN plays a pivotal role in modeling global and nonlocal dependencies, particularly in spectrally mixed or topologically complex scenes such as IP. Meanwhile, the S^2CsN module provides essential fine-scale spatial representations, significantly enhancing edge sensitivity and boundary localization across all datasets. In addition, the SFPN module improves the quality of the input features by increasing spectral discriminability, thereby facilitating more effective downstream convolutional learning. Overall, the complete GTCFN architecture demonstrates strong intermodule synergy. The integration of spectral, spatial, and relational modeling branches leads to consistent and substantial performance gains across all evaluation metrics, reinforcing the importance of unified spatial–spectral–topological representation learning in HSIC.

B. Analysis of Training Samples

To further evaluate the robustness and generalization ability of the proposed GTCFN model, we conducted experiments under limited-label scenarios. Specifically, we randomly selected a varying number of labeled samples per class—[5, 10, 15, 20, 25, 30]—on each dataset and assessed the model's performance in comparison with multiple advanced methods. The comparative results are illustrated in Fig. 14.

GTCFN consistently outperforms other methods across different datasets and sample sizes. These experiments collectively demonstrate three key strengths of the proposed GTCFN model. First, it exhibits strong few-shot learning capabilities, achieving competitive accuracy with very limited labeled samples. Second, it shows stable performance improvement as the number of training samples increases. Third, it consistently delivers top-tiered results across datasets with varying complexity—including rural and urban scenes, and both regular and irregular class structures. Overall, GTCFN maintains high classification accuracy and strong generalization ability across different datasets and sample sizes by effectively integrating hyperpixel-based structural encoding, lightweight graph-based reasoning, and multiscale spectral–spatial feature fusion.

C. Complexity Analysis

To evaluate the computational complexity of GTCFN, we conducted experiments on four benchmark datasets using 30 labeled samples per class, summarized in Table XI. The comparison includes both training and testing time, as well as model parameter size. Overall, GTCFN exhibits the highest efficiency among all competing methods, consistently achieving the shortest training and testing times across datasets. Specifically, on the IP dataset, GTCFN requires only 8.07 s for training and 0.48 s for testing, significantly outperforming methods such as S^2Mamba (145.32/9.14 s), HSI-MFormer (124.17/2.42 s), SSRN (105.24/3.39 s), and GTFN (121.42/18.35 s). A similar trend is observed on the PU dataset, where GTCFN reduces the testing time to 4.54 s, which is one order of magnitude faster than HSI-MFormer (8.96 s) and the most accurate method WFCG (16.45 s). On the SA dataset, GTCFN maintains rapid testing (2.75 s) with minimal training overhead. On the most computationally intensive

HONGHU dataset, it requires only 13.52 s for testing, whereas WFGC and AMGCFN exceed 60 and 50 s, respectively. In terms of model complexity, GTCFN also maintains a relatively lightweight architecture with only 234 708 parameters, which is significantly fewer than most competing approaches. This combination of high computational efficiency and competitive accuracy highlights GTCFN's practical applicability in real-world scenarios, where time and computational resources are limited. To evaluate the computational complexity of GTCFN, we conducted experiments on four benchmark datasets using 30 labeled samples per class, summarized in Table IV.

VI. Conclusion

In this article, we propose a novel GTCFN for HSIC. The architecture effectively unifies the strengths of CNNs in capturing local spatial–spectral correlations and the global modeling capabilities of graph-based Transformers. By jointly employing an S^2CsN and a GTsN, GTCFN effectively learns complementary pixel-level fine-grained patterns and superpixel-level structural semantics. This dual-branch design enables the model to capture long-range dependencies while preserving graph structural information, thereby enhancing classification performance under heterogeneous and limited-sample conditions. To further improve computational efficiency and model robustness, GTCFN introduces several key innovations: a kernelized attention mechanism using random feature mapping significantly reduces the quadratic complexity of standard self-attention, while a Gumbel-based multihead stochastic aggregation mechanism mitigates over-smoothing and enhances generalization. Extensive experiments on four widely used hyperspectral datasets have demonstrated that the GTCFN is consistently best-in-class in terms of OA, AA, and Kappa coefficients. Notably, the model exhibits strong robustness under label-limited conditions and generalizes well across both structured urban layouts and spectrally mixed rural regions.

In future work, we intend to extend GTCFN to semi-supervised and unsupervised learning paradigms, incorporating recent self-supervised graph representation techniques [52] to reduce annotation dependence and enhance cross-domain adaptability in real-world remote sensing applications. We will also focus on improving the model's flexibility and generalization through dynamic graph construction, adaptive sampling strategies, and automated graph learning mechanisms. These improvements are expected to further facilitate the development of HSIC.

References

[1] Q. Tong, B. Zhang, and L. Zhang, "Frontier progress in hyperspectral remote sensing in China," *J. Remote Sens.*, vol. 20, no. 5, pp. 689–707, 2016.

[2] H. Chen, J. Gan, and Q. Liu, "Application and prospect of hyperspectral remote sensing technology in geoenvironmental monitoring," *Urban Geol.*, vol. 19, no. 2, pp. 176–183, 2024.

[3] X. Zhang, "Application of remote sensing technology in agricultural production," *Agricult. Sci. Inf.*, vol. 51, no. 8, pp. 35–38, 2023.

[4] Y. Zhang, Y. Guo, R. Tang, and Z. Tang, "Progress and trends in the application of hyperspectral remote sensing in plant diversity research," *J. Remote Sens.*, vol. 27, no. 11, pp. 2467–2483, 2023.

[5] J. Zhao et al., "Characterization of alteration of Gansu huaxishan gold mine based on hyperspectral data and application to mineral search prediction," *Geol. Explor.*, vol. 59, no. 1, pp. 122–133, 2023.

[6] Y. Ma, W. Zhang, and D. Liu, "Characterization of hyperspectral reconnaissance technology and analysis of its threat to military targets on the ground," *Shanghai Aerosp. Corp.*, vol. 29, no. 1, pp. 37–40, 2012.

[7] W. Ma et al., "Hyperspectral image classification based on spatial and spectral kernels generation network," *Inf. Sci.*, vol. 578, pp. 435–456, Nov. 2021.

[8] S. S. Sawant and M. Prabukumar, "A survey of band selection techniques for hyperspectral image classification," *J. Spectral Imag.*, vol. 9, no. 1, p. 5, 2020.

[9] X. Kang, X. Xiang, S. Li, and J. A. Benediktsson, "PCA-based edge-preserving features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 12, pp. 7140–7151, Dec. 2017.

[10] T. V. Bandos, L. Bruzzone, and G. Camps-Valls, "Classification of hyperspectral images with regularized linear discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 3, pp. 862–873, Mar. 2009.

[11] B. Schölkopf, A. J. Smola, and K. Müller, "Kernel principal component analysis," in *Proc. Int. Conf. Artif. Neural Netw.*, 1997, pp. 583–588.

[12] S. Delalieux, B. Somers, B. Haest, T. Spanhove, J. Vanden Borre, and C. A. Mücher, "Heathland conservation status mapping through integration of hyperspectral mixture analysis and decision tree classifiers," *Remote Sens. Environ.*, vol. 126, pp. 222–231, Nov. 2012.

[13] Y. Zhang, G. Cao, X. Li, and B. Wang, "Cascaded random forest for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 4, pp. 1082–1094, Apr. 2018.

[14] C. Bo, H. Lu, and D. Wang, "Spectral–spatial K-nearest neighbor approach for hyperspectral image classification," *Multimedia Tools Appl.*, vol. 77, no. 9, pp. 10419–10436, May 2018.

[15] S. Zhong, C.-I. Chang, and Y. Zhang, "Iterative support vector machine for hyperspectral image classification," in *Proc. 25th IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2018, pp. 3309–3312.

[16] M. Ahmad, A. Khan, M. Mazzara, and S. Distefano, "Multi-layer extreme learning machine-based autoencoder for hyperspectral image classification," in *Proc. 14th Int. Joint Conf. Comput. Vis., Imag. Comput. Graph. Theory Appl.*, 2019, pp. 75–82.

[17] G. E. Hinton, S. Osindero, and Y.-W. Teh, "A fast learning algorithm for deep belief nets," *Neural Comput.*, vol. 18, no. 7, pp. 1527–1554, Jul. 2006.

[18] Z. Lin, Y. Chen, X. Zhao, and G. Wang, "Spectral–spatial classification of hyperspectral image using autoencoders," in *Proc. 9th Int. Conf. Inf., Commun. Signal Process.*, Dec. 2013, pp. 1–5.

[19] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[20] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[21] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, no. 1, 2015, Art. no. 258619.

[22] A. Ben Hamida, A. Benoit, P. Lambert, and C. Ben Amar, "3-D deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.

[23] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[24] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[25] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[26] X. Zhang, Y. Su, L. Gao, L. Bruzzone, X. Gu, and Q. Tian, "A lightweight transformer network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5517617.

[27] J. Zhang, Y. Zhang, and Y. Zhou, "Multiview transformer: Rethinking spatial information in hyperspectral image classification," 2023, *arXiv:2310.07186*.

[28] S. Cheng, R. Chan, and A. Du, "CACFTNet: A hybrid cov-attention and cross-layer fusion transformer network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 10460571.

[29] A. Yang, M. Li, Y. Ding, D. Hong, Y. Lv, and Y. He, "GTFN: GCN and transformer fusion network with spatial–spectral features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 6600115.

[30] Y. Ding et al., "Adaptive homophily clustering: Structure homophily graph learning with adaptive filter for hyperspectral image," 2025, *arXiv:2501.01595*.

[31] Y. Zhou, H. Zheng, X. Huang, S. Hao, D. Li, and J. Zhao, "Graph neural networks: Taxonomy, advances, and trends," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 1, pp. 1–54, Feb. 2022.

[32] J. Li et al., "Superpixel-based dual-neighborhood contrastive graph autoencoder for deep subspace clustering of hyperspectral image," in *Proc. Int. Conf. Intell. Comput.*, 2024, pp. 181–192.

[33] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 139–144.

[34] F. F. Shahraki and S. Prasad, "Graph convolutional neural networks for hyperspectral data classification," in *Proc. IEEE Global Conf. Signal Inf. Process. (GlobalSIP)*, Nov. 2018, pp. 968–972.

[35] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[36] L. Mou, X. Lu, X. Li, and X. X. Zhu, "Nonlocal graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 12, pp. 8246–8257,Dec. 2020.

[37] Y. Ding, X. Zhao, Z. Zhang, W. Cai, N. Yang, and Y. Zhan, "Semi-supervised locality preserving dense graph neural network with ARMA filters and context-aware learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–12, 2022.

[38] Z. Ma, Z. Jiang, and H. Zhang, "Hyperspectral image classification using feature fusion hypergraph convolution neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2022.

[39] S. Wan, C. Gong, P. Zhong, S. Pan, G. Li, and J. Yang, "Hyperspectral image classification with context-aware dynamic graph convolutional network," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 597–612, Jan. 2021.

[40] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.

[41] H. Zhou, F. Luo, H. Zhuang, Z. Weng, X. Gong, and Z. Lin, "Attention multihop graph and multiscale convolutional fusion network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5508614.

[42] Y. Wang, Z. Xue, M. Jia, Z. Liu, and H. Su, "Hypergraph convolutional network with multiple hyperedges fusion for hyperspectral image classification under limited samples," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5526318.

[43] C. Ding et al., "Integrating prototype learning with graph convolution network for effective active hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5504816.

[44] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 1–9.

[45] H. Yan, E. Zhang, J. Wang, C. Leng, A. Basu, and J. Peng, "Hybrid conv-ViT network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.

[46] Q. Wu, W. Zhao, Z. Li, D. Wipf, and J. Yan, "NodeFormer: A scalable graph structure learning transformer for node classification," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 27387–27401.

[47] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[48] X. Zhao et al., "MGHT: A multi-granularity hyperspectral image classification method based on hybrid transformer," *Spectrosc. Lett.*, pp. 1–17, Mar. 2025.

[49] Y. Dong, Q. Liu, B. Du, and L. Zhang, "Weighted feature fusion of convolutional neural network and graph attention network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 1559–1572, 2022.

[50] G. Wang, X. Zhang, Z. Peng, T. Zhang, and L. Jiao, "S^2mamba: A spatial–spectral state space model for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5511413.

[51] Y. He, B. Tu, B. Liu, J. Li, and A. Plaza, "HSI-MFormer: Integrating mamba and transformer experts for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5621916.

[52] Y. Ding et al., "SLCGC: A lightweight self-supervised low-pass contrastive graph clustering network for hyperspectral images," 2025, *arXiv:2502.03497*.